



University of Bologna
Dipartimento di Informatica –
Scienza e Ingegneria (DISI)
Engineering Bologna Campus

Class of ICCBD
Infrastructures for
Cloud Computing and Big Data M

ONs and Advanced Filesystems

Antonio Corradi
Academic year 2025/2026

OVERLAY NETWORKS

There are many situations where you want a **logical connection among different very far entities spread over different locations and networks**

The solution is an **Overlay Network (ON)** at the **application level**
An ON is a network that connects at high level (application) all those entities to be considered together a group

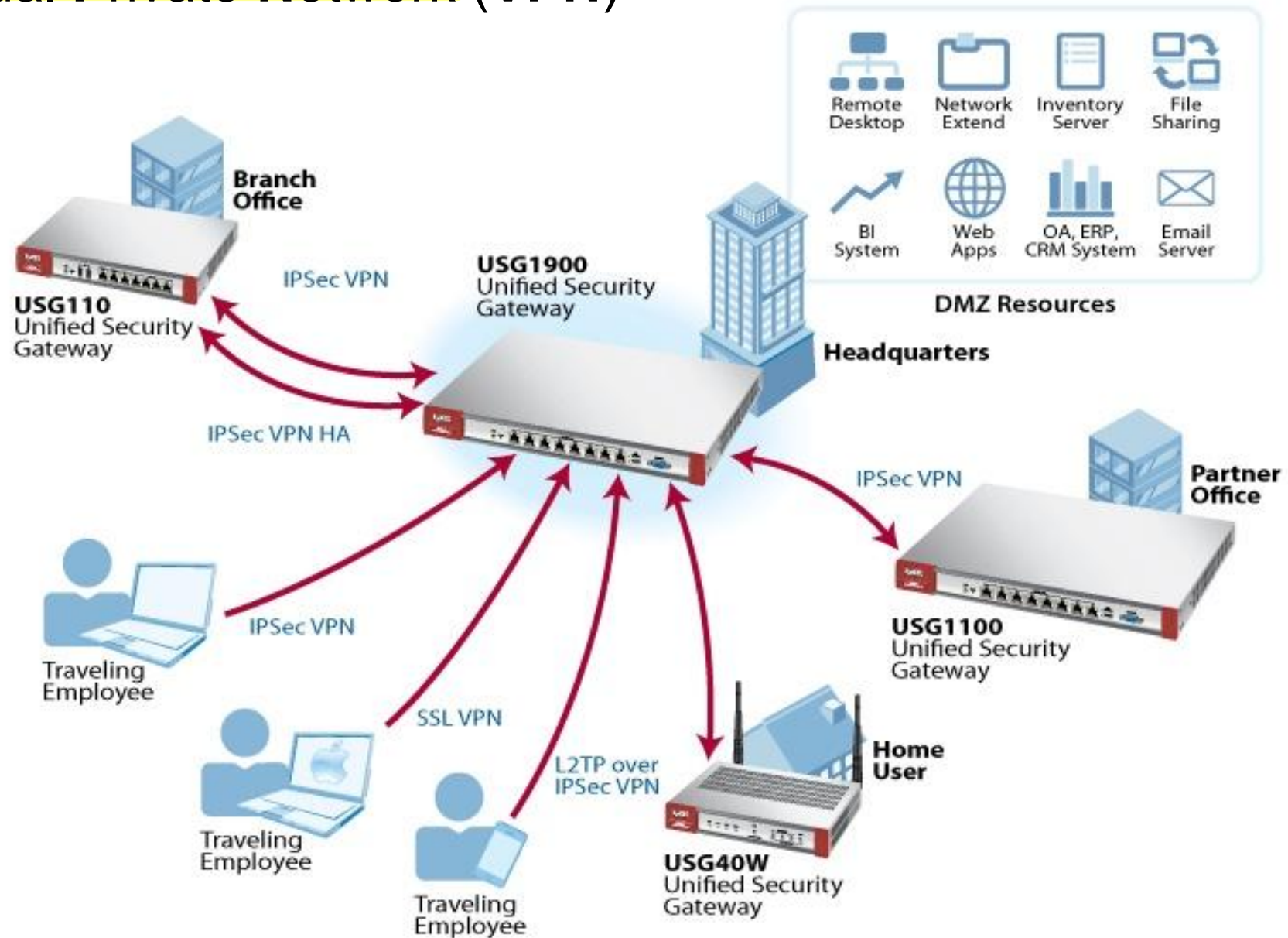
Overlay networks may *have very different goals and also enforced in different ways*, but their importance is paramount in many situations because **they answer the core requirements of efficiency, dynamicity, and scalability**

One main point very important is not only **organizing** it, but also to **grant QoS** and to **respect an agreed SLA**

That is why there are **many different solutions** for different cases, and many different tools interested in these requirements (look for mobile IPs, current locations, Message Oriented Middlewares, ...)

OVERLAY NETWORK EXAMPLE

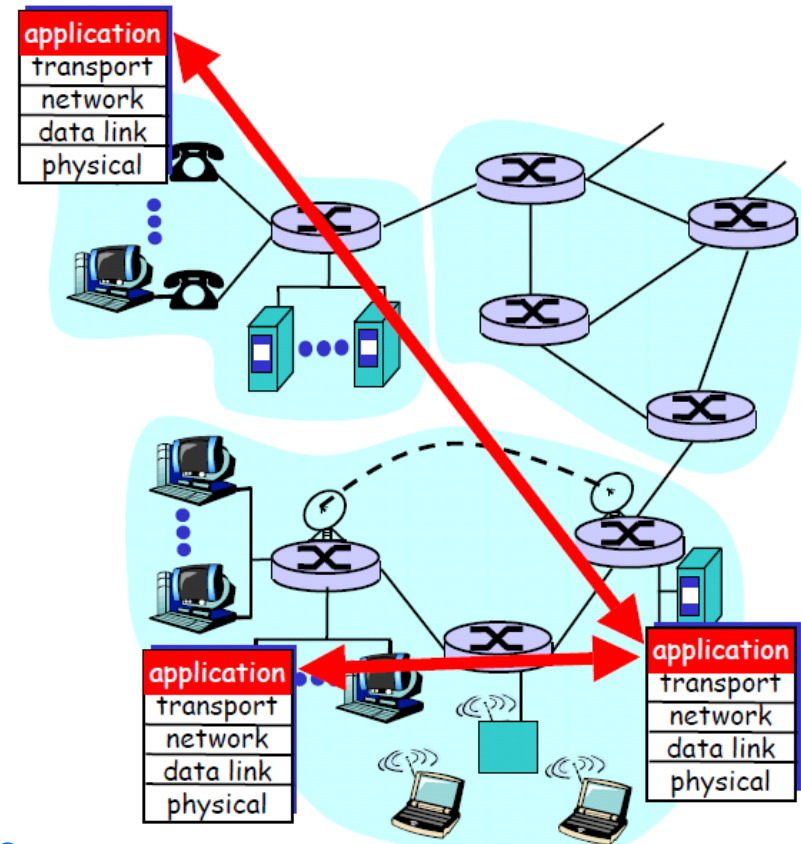
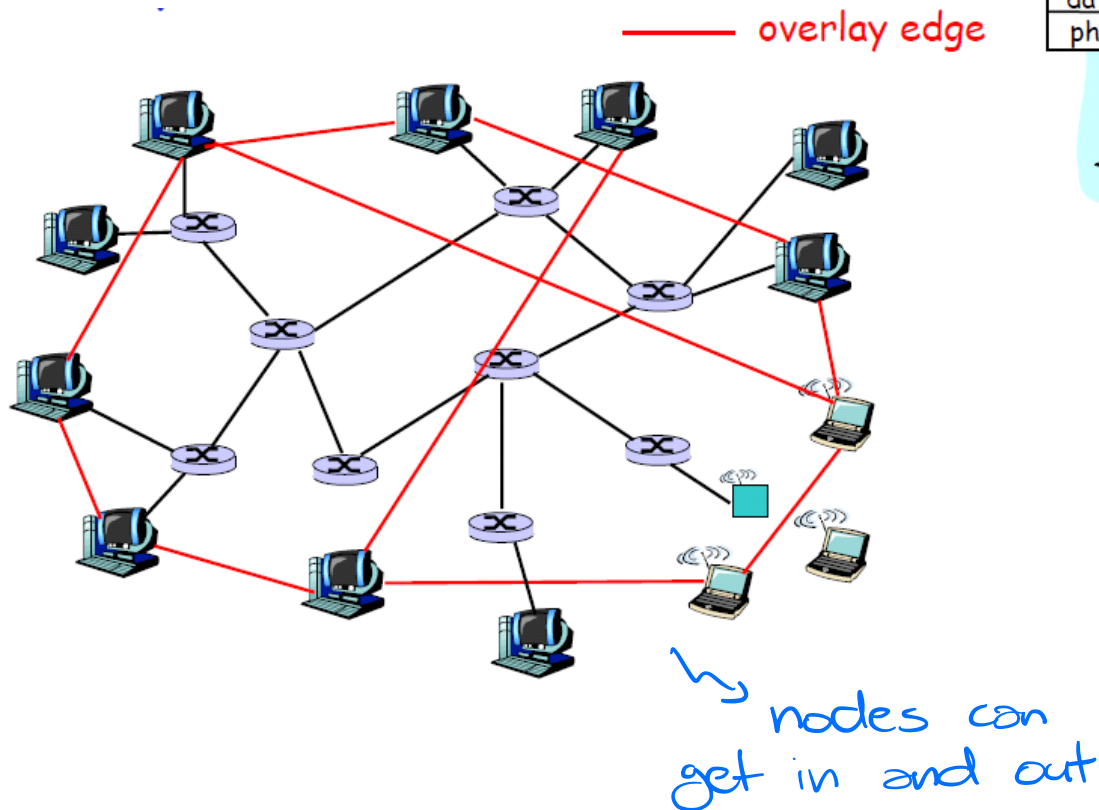
Virtual Private Network (VPN)



OVERLAY NETWORKS ONs

The main point is both to create a **new network at the application level** and to **maintain it** with **specified requirements**

All participants become part of it and **can freely communicate** (the same as *if they had a real network connection*), by using an **application neighborhood**



CLASSIFICATION OF OVERLAY NETWORKS

There are two main different kinds:

- **Unstructured overlays**
- **Structured overlays**

By focusing on new nodes arriving and entering the ON,
in **Unstructured overlays**, new nodes choose randomly the neighbor to use to access to the ON

in **Structured overlays**, there is a precise strategy to let nodes in and to organize the architectures, maintained also to react to discontinuities and failures

ONs propose solutions for P2P applications, but also for MOMs (even if they are statically-oriented)

P2P

Napster, Gnutella, Kazaa, BitTorrent

Support

Chord, Pastry/Tapestry, CAN

Social Nets

MSN, Skype, Social Network Support

OVERLAY NETWORK: USAGE

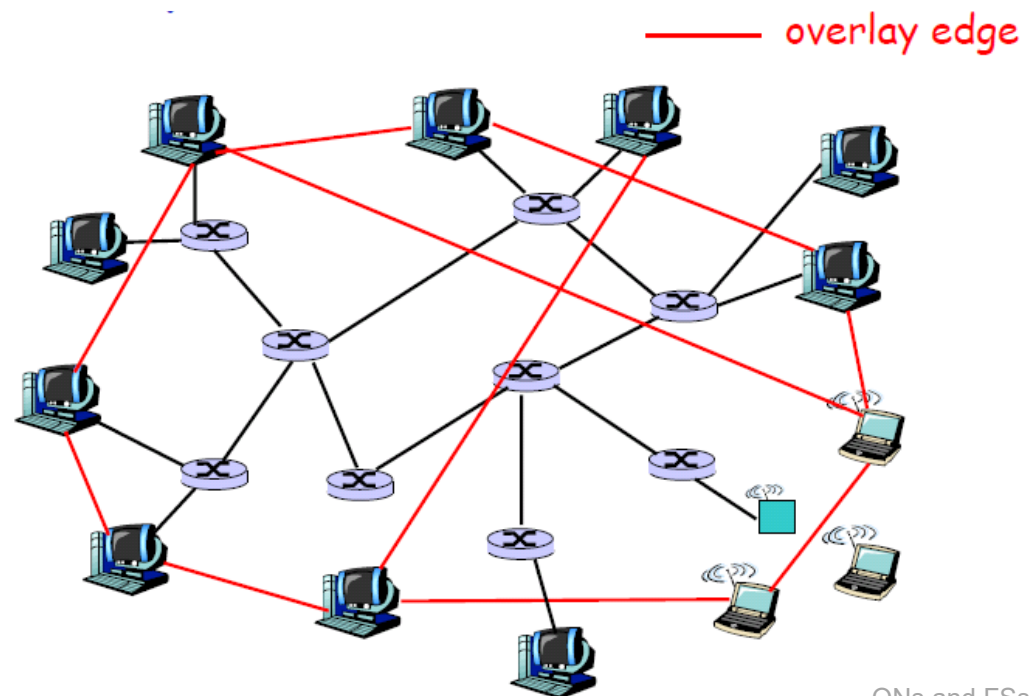
A good **overlay network** has the goal of making more **efficient the operations among the group of current participants** answering some **specific requirements**

All participants in an overlay have **a common goal of exchanging information**, for instance...

they may intend to exchange data: files in a P2P application, messages in social nets, specific application protocols in other environments, etc.

The **nodes** should:

- **get in**
- **make its actions**
- **help actions of others**
- **get out**



SYSTEM AND APPLICATION KEY ISSUES

ONs should organize the communication support and also enable the application-level management

Lookup (application) *~> get in*

- To find out **very fast the appropriate user information** (content/resource) on the ON

Guaranteed Throughput (support)

- To communicate **over an ON** needs **support** for content **distribution/dissemination**
- To **replicate content** ... fast, efficiently, reliably

Management (support)

- To **maintain efficiently** the ON under a **high rate of connections/disconnections** and intermittent failures in load balanced approach
- To **guarantee both application reliability and availability** (maybe very difficult): a **self-organizing** approach is typically followed

ON MANAGEMENT PROPERTIES

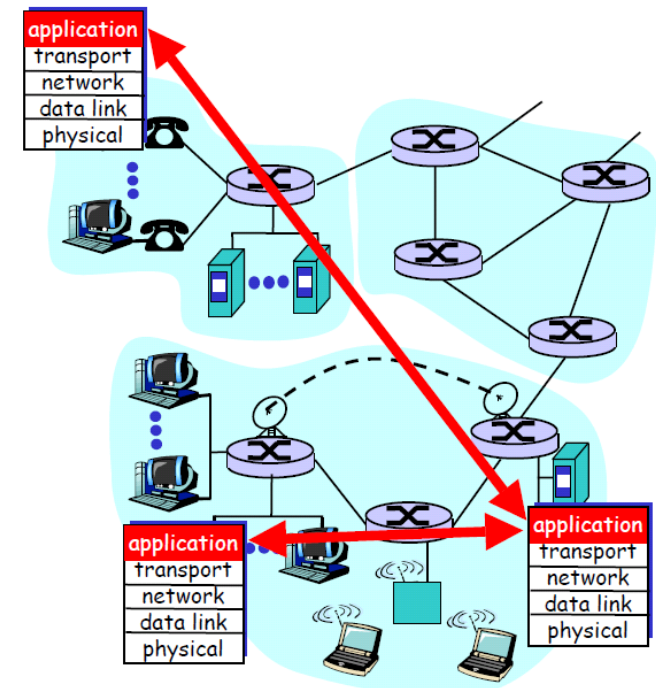
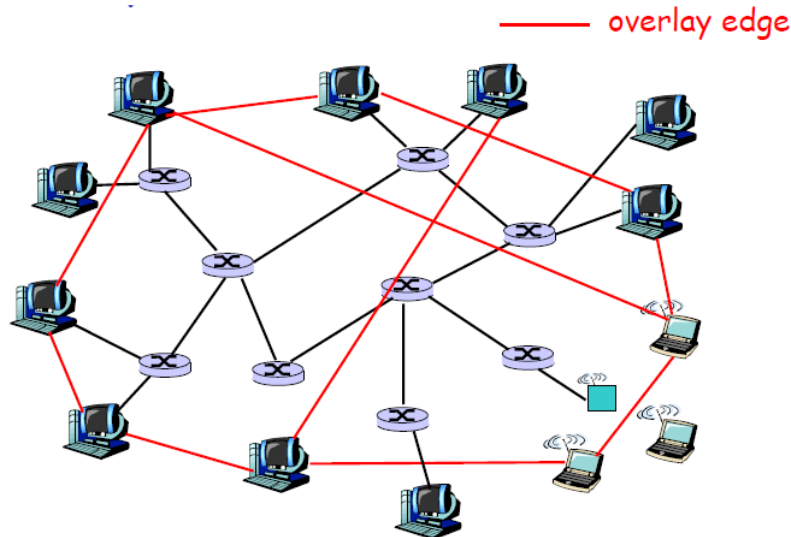
Overlay networks imply many challenges to cope with, while executing ON operations

- **Maintaining** the edge links (via pointers to IP addresses?)
- **Favoring** the insertion in the neighborhood
- **Checking** link liveness
- **Identifying** problems and faults
- **Recovering** edges
- **Overcoming** nodes going down and their unavailability
- **Re-organizing the overlay**, when some nodes leave the network and other nodes get in
- **Keeping the structure**, despite mobile nodes intermittent presence (and eventual crashes or leaving)
- **Creating a robust connection**, independently of omissions and crashes (QoS?)

OVERLAY NETWORKS PROPERTIES

There two main properties ON must comply with their QoS:

- **Dynamicity** of supporting nodes that can get int and out (even by crashes)
- **Replication of resources** to make them available anyway (independent on any event pattern on ON)



EX - NAPSTER: A PIONEER P2P (1991)

A non-structured approach for file retrieving

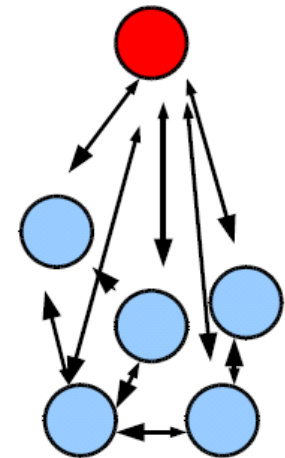
Centralized Lookup → single point of failure (congestion)

One Centralized directory services deal with nodes entering

- Any node connects to a Napster server
- Any node uploads list of files to server
- Any node gives servers keywords to search the full list with

File exchange peer to peer

- Lookup is centralized from servers
- but files copied P2P
- Select “best” of correct answers
- (announce by ping messages)



Performance Bottleneck of Directory and consequent Low scalability

GNUTELLA (2000)

GNUTELLA is the main representative of **unstructured ONs**, by providing a **distributed approach** in **file retrieval**

Any node entering GNUTELLA tries to connect to some others locally available

Fully decentralized organization and **lookup** for files

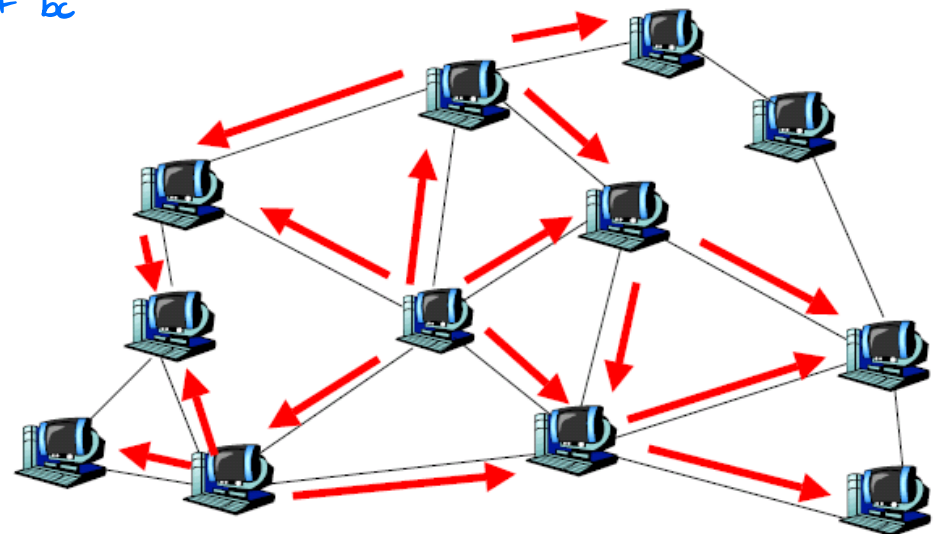
- There are nodes with different degrees of connections and availability (from **high-degree** nodes to **low-degree** ones)

High-degree nodes may receive and control even more links

→ it's not a multicast all over the Internet bc a time to live is specified

- Flooding based lookup**, obviously **inefficient** in terms of scalability and bandwidth

Limiting the cost as much as possible



GNUTELLA GET IN VIA NEIGHBORHOOD

Step 0: *Join the network*

Step 1: *Determining who is on the network*

- "Ping" packet is used to announce your presence on the network.
- Other peers respond with a "Pong" packet and Ping connected peers
- A Pong packet also contains:
 - IP address, port number, amount of data that peer share
 - Pong packets come back via same route of Ping

Step 2: *Searching*

- Gnutella "Query" ask other peers (N usually 7) for **desired files**
- A Query packet might ask, *"Do you have any matching content with the string 'Volare'?"*
- Peers check to see if they have matches & respond (if they have any match) & send packet to connected peers if not (N usually 7)
- It continues for **TTL** steps or Time-To-Live (T specifies the number of hops a packet can traverse before dying, typically 10)

GNUTELLA (NEIGHBOR SCENARIO)

Step 3: *Downloading*

- Peers respond with a “QueryHit” (it contains contact info)
- File transfers via direct connection by using HTTP protocol - GET method

REACHABILITY and COST

An analytical estimation of reachable users

(*T* - # of hop propagation and *N* - # of neighbors)

T : TTL, *N* : Neighbors for Query

	<i>T=1</i>	<i>T=2</i>	<i>T=3</i>	<i>T=4</i>	<i>T=5</i>	<i>T=6</i>	<i>T=7</i>
<i>N=2</i>	2	4	6	8	10	12	14
<i>N=3</i>	3	9	21	45	93	189	381
<i>N=4</i>	4	16	52	160	484	1,456	4,372
<i>N=5</i>	5	25	105	425	1,705	6,825	27,305
<i>N=6</i>	6	36	186	936	4,686	23,436	117,186
<i>N=7</i>	7	49	301	1,813	10,885	65,317	391,909
<i>N=8</i>	8	64	456	3,200	22,408	156,864	1,098,056

GNUTELLA SEARCH

GNUTELLA different versions adopted different scalability protocols

Flooding based search is extremely wasteful in bandwidth

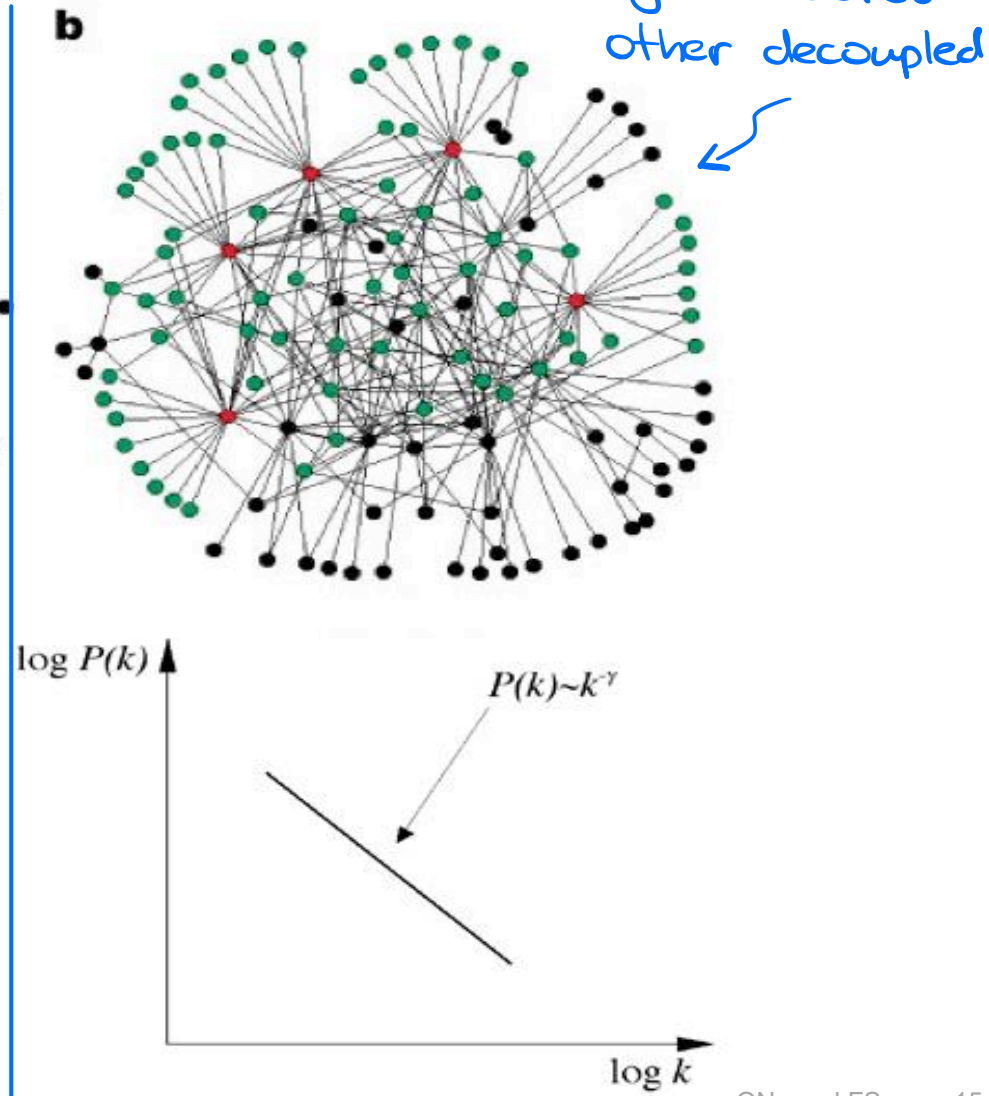
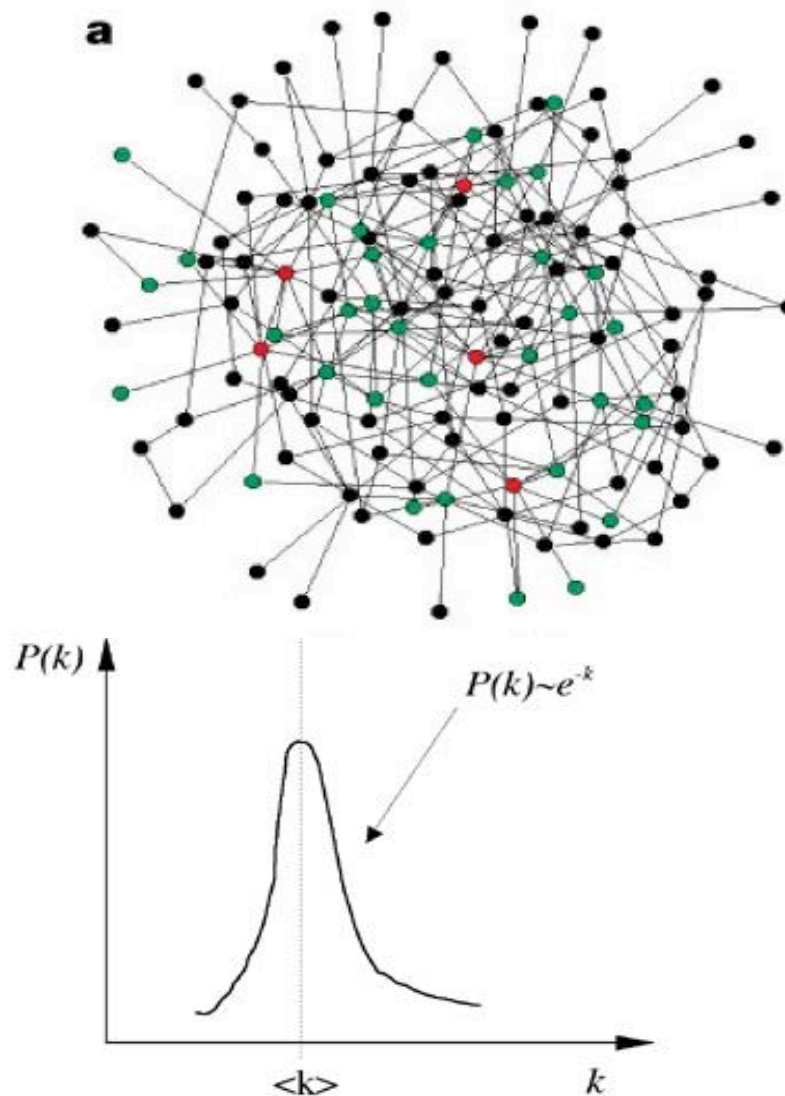
- Enormous number of **redundant messages** (not efficient)
- A large (linear) part of the network is covered irrespective of hits found, **without taking into account needs**
- All users do **searches in parallel**: local load grows linearly with size

Taking advantage of the unstructured network, some more efficient protocols started appearing

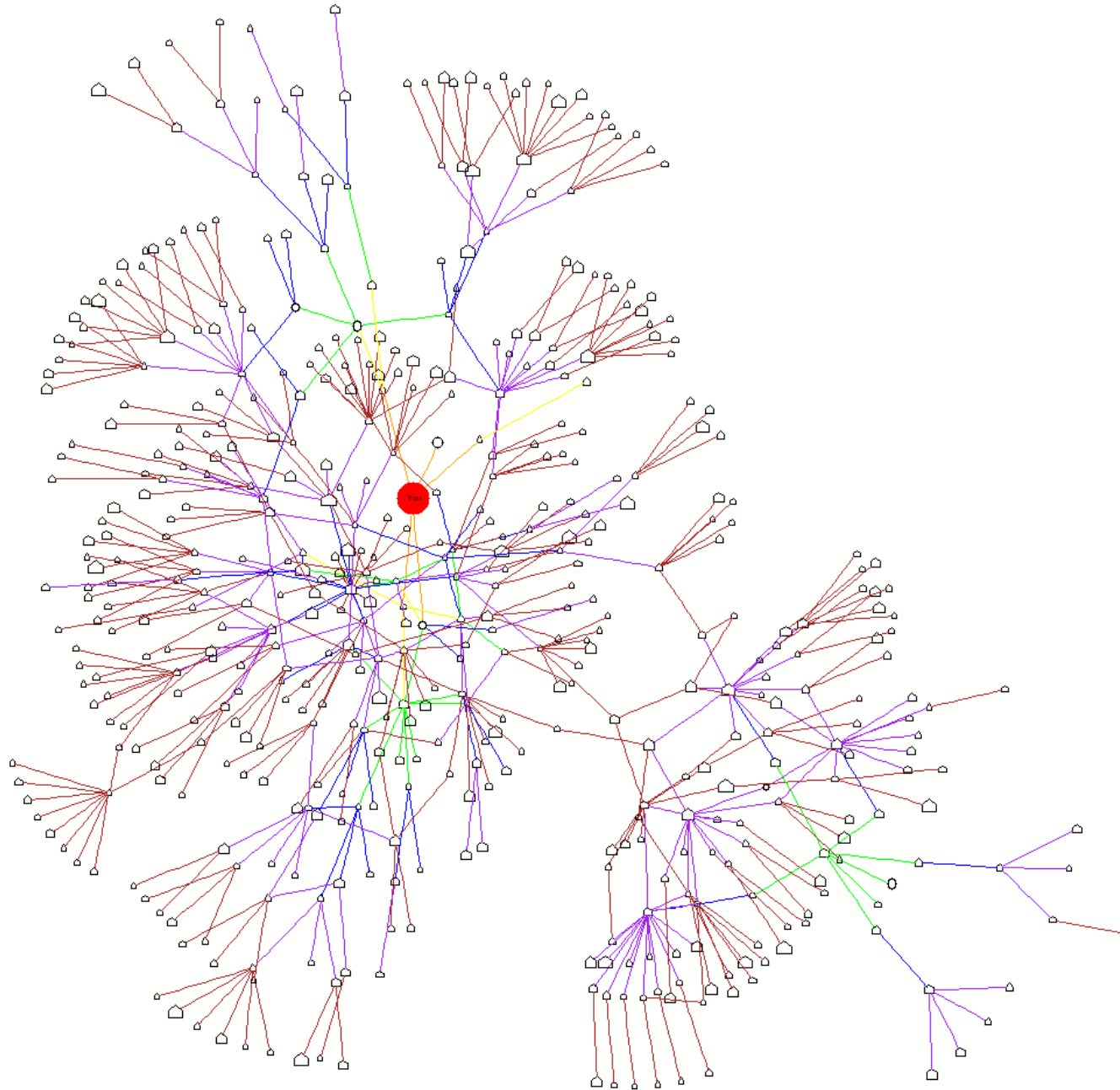
- **Controlling topology** for better search
Random walk, Degree-biased Random Walk
- **Controlling placement of objects**
Replication

RANDOM VS SCALE-FREE NETWORKS

a **Scale-Free net** (b - on the right) is very different from a randomly connected network (a - left curve)



GNUTELLA NODES



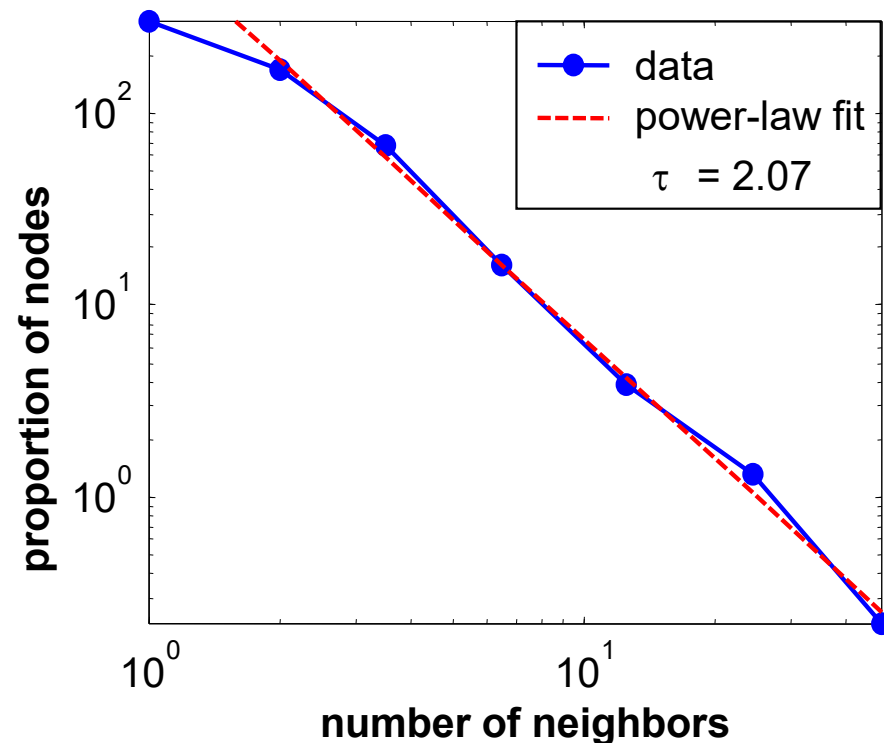
GNUTELLA NODES

A **Scale-Free graph** is a graph whose degree of distribution follows a **power law** or an **exponential law**

Nodes behave very different: a few are highly connected nodes and many low connected ones

Basic strategy based on **high degree** nodes

High-degree nodes can store the index about a **large portion of the network** and are easier to find by (**biased**) **random walk** in a **scale-free graph** in a scenario of random offer of files



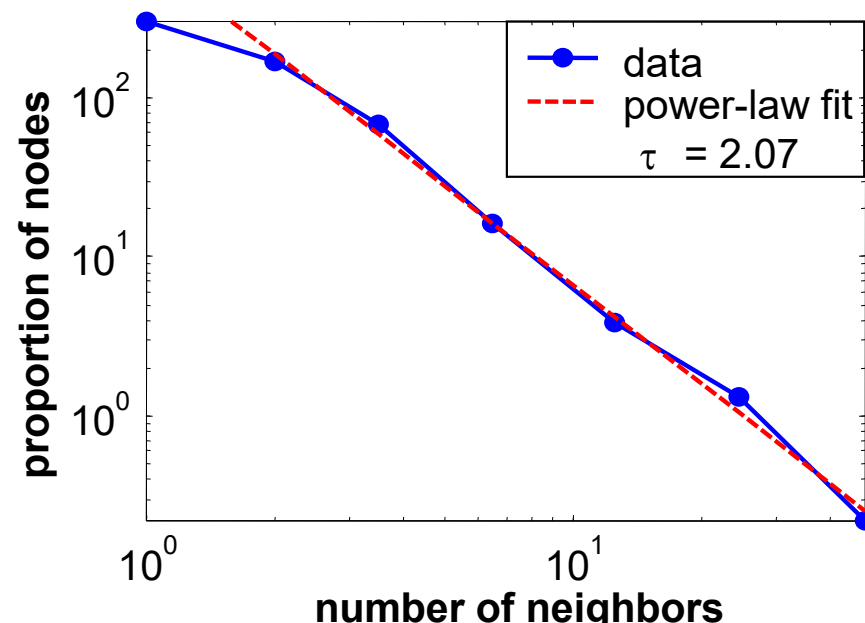
GNUTELLA NODES

High-degree nodes (**hub nodes**) have a largest neighborhood of low-degree ones

Random walk (moves random to avoid visiting always the same already last visited nodes)

Degree-biased random walk

- Select **highest degree nodes or hubs** that have not been visited
- Walk **first climbs** to highest degree nodes, then **climbs down** on vicinity
- Optimal coverage can be **formally** proved



GNUTELLA REPLICATION

The main idea is to **spread copies of objects to peers** so that **more popular objects can be found easier** and **also launch more walks in parallel to more likely find them**

Replication is both in sense of **more copies of data** and also in terms of **more walkers to launch in parallel**

Replication strategies

*Replicate with i when q_i is the **number of queries** for object i*

Owner replication

- Produce replicas in proportion to q_i

Path replication

- Produce **replicas over the path** with replication as square root to q_i

Random replication

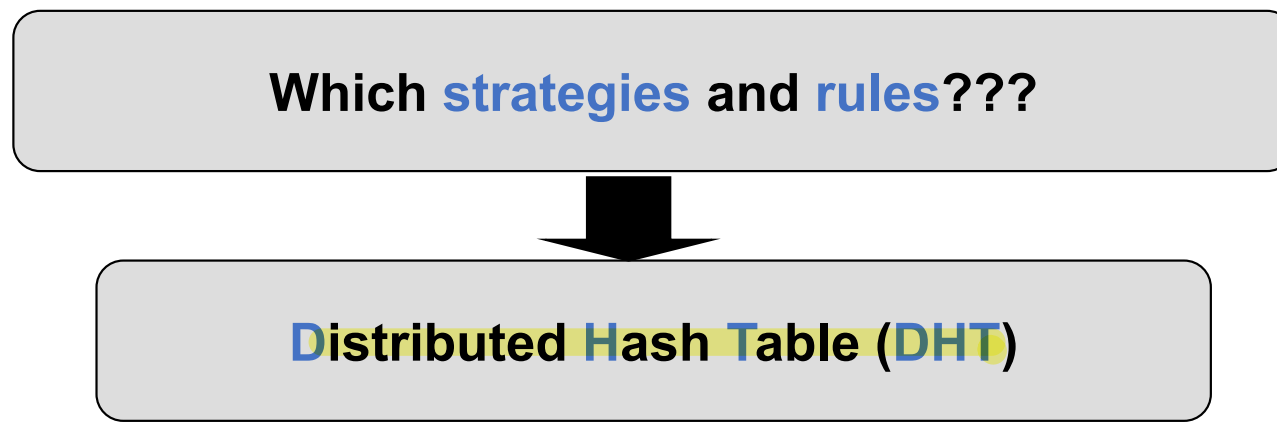
- Same as path replication to q_i , only using the given number of **random nodes, not the path**

...but it is still **difficult to find rare objects**

UNSTRUCTURED VS STRUCTURED

To go deep into ON organization...

- **Unstructured P2P networks** allow resources to be placed at any node **spontaneously**
The network topology is arbitrary and the growth is free but some worst cases and bottlenecks
- **Structured P2P networks** simplify **resource location** and **load balancing**, by defining a **topology** and **rules** for **resource placement** to obtain **efficient search for rare objects**



HASH TABLES

Distributed Hash Tables use **Hash principles** toward a better retrieval of data **content** and **value** in a distributed setting

Store **arbitrary keys** and **connected data** (value)

- **put** (key, value)
- value = **get**(key)

Lookup must be **fast**

- Calculate **hash function** $h()$ on **key** that returns a **storage cell**

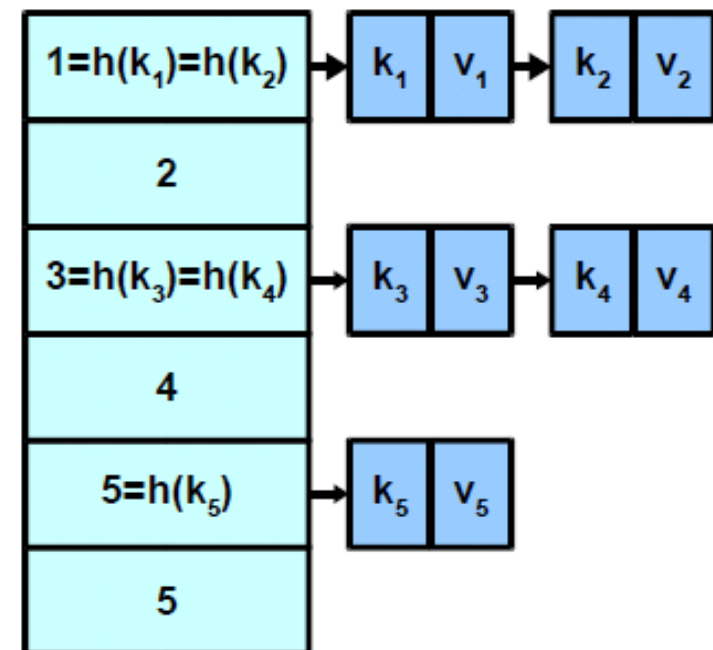
Chained hash table

- Store **keys** in the **overflow chain** (together with optional value)

Nodes must support the storing of values

Allocated array:
indexed by hash
values

Stored entries

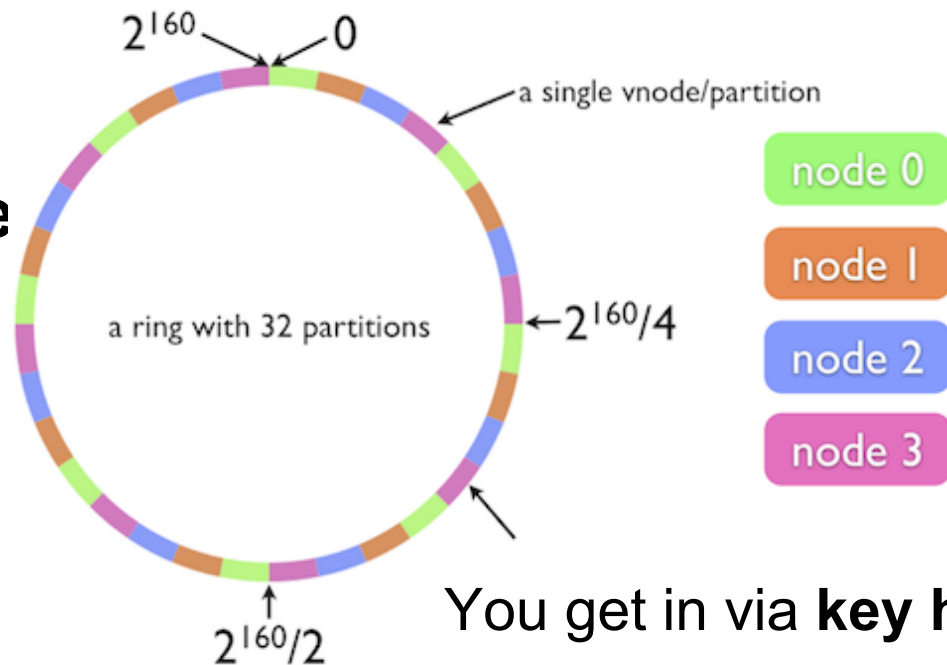


DISTRIBUTED HASH TABLES

You can **partition the whole space of keys** via a ring in which you have **different containing nodes** (4 nodes here)

Nodes can store data to be retrieved via key (1-4)

**You must
decide where
to put data
and how to
get data fast**



You get in via **key hashing**

Important properties: **nodes easy get in, often get out**

- **Safety of data** (**do not loose any information**)
- **Adaptation to changes** (**nodes get in / get out**)

DISTRIBUTED HASH TABLES

Hash table functions for ON are typically P2P: the **lookup of data indexed by keys can be very efficient and fast** (**find the nodes where the data are kept**)

Key-hash → node mapping

- Assign a **unique live node to any key**
- **Find this node quickly and cheaply** in the overlay network
- **Query, presented to different nodes, can work in parallel over different nodes** (in a ring)

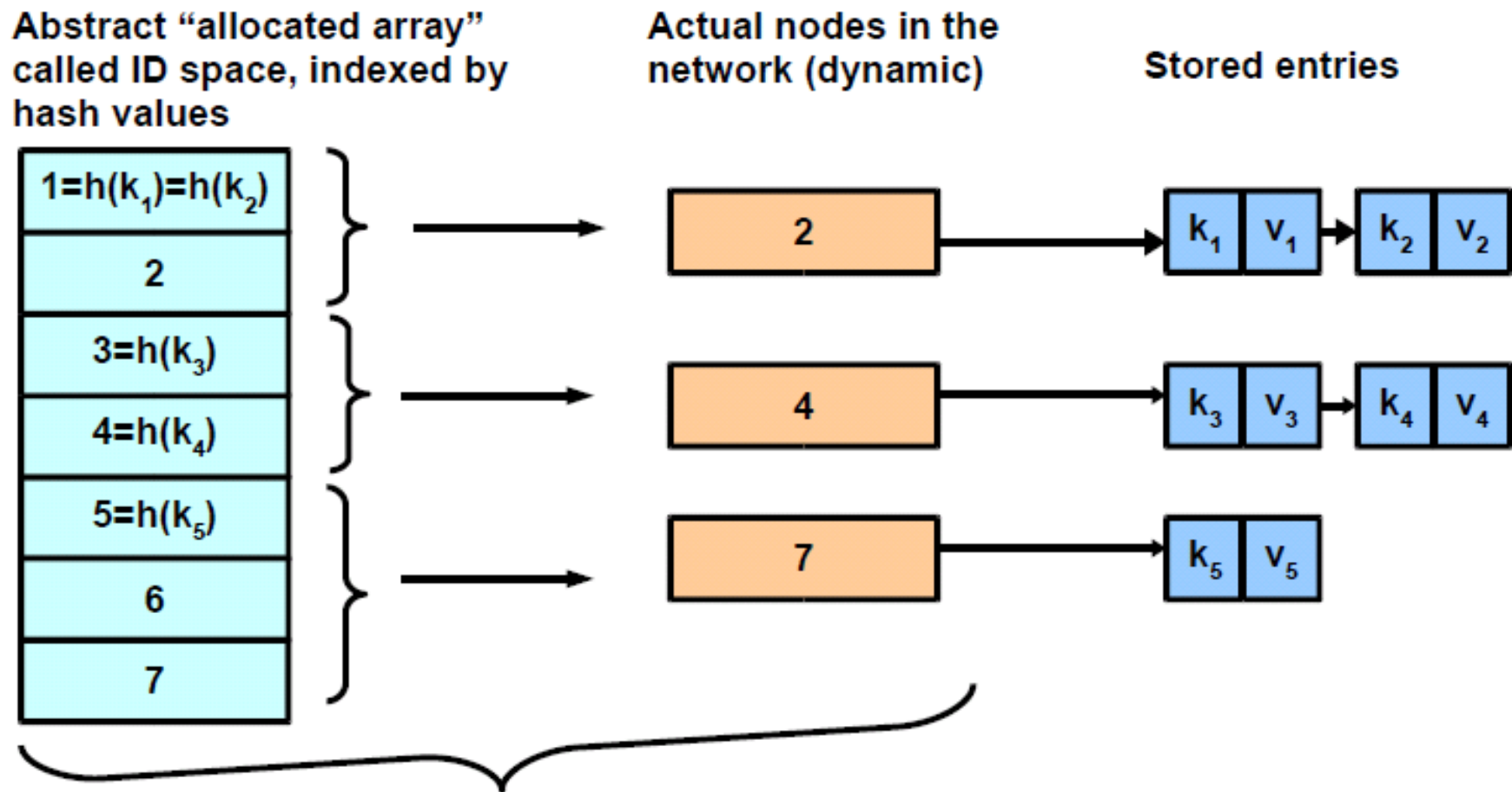
Again, **some forms of replication** must be decided to avoid to **lose data but to maintain copies**, in case one or more nodes crash

Partitioning and Replication

As main organization principles

DISTRIBUTED HASH TABLES

Find the **best node allocation** depending on **existing nodes** where nodes can enter and leave the ON (now 2,4,7 nodes)



consistent hashing of keys to nodes
typically two step, as shown above

DISTRIBUTED HASH TABLES

Support maintenance of the **ON** and **optimization of its current organization of nodes**

- **Load balancing**: maybe even change the key-hash when the nodes change (**dynamcity of nodes**) → necessity of **node mapping** on the fly
- **Replicate entries on more nodes** to increase availability by using **neighbor close nodes**

STRUCTURED HASH TABLES

Many examples of tools for supporting Distributed Hash Tables - DHT

Chord (2001)

Consistent hashing **ring-based** structure

Pastry (2001)

Uses an **ID space** concept as in Chord

but exploits the concept of a **nested group** for **acceleration**

Many other DHT solutions proposed

CAN

Nodes/objects are mapped into a d-dimensional cartesian space

...

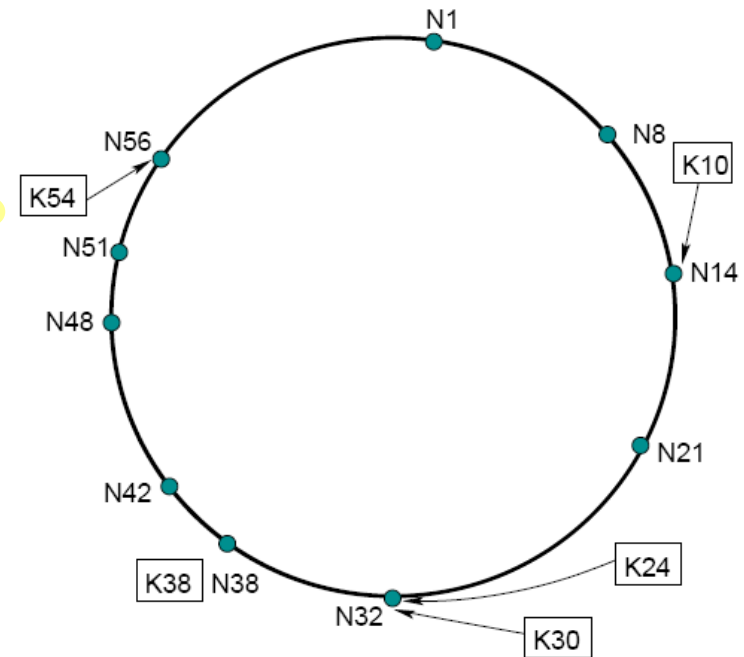
CHORD HASH TABLES

Hash is applied over the name space of a dynamic ring that connects all available nodes who contains values

Consistent hashing based on an ordered ring overlay of the nodes in the name space of keys

N nodes – K keys

- Both keys and nodes are hashed to 160 bits IDs (SHA-1 Secure Hash Algorithm 1) from 0 to $2^{160}-1$ bits
- Keys are assigned to nodes by using **consistent hashing**
 - The key is stored into the successor node available in the ID space



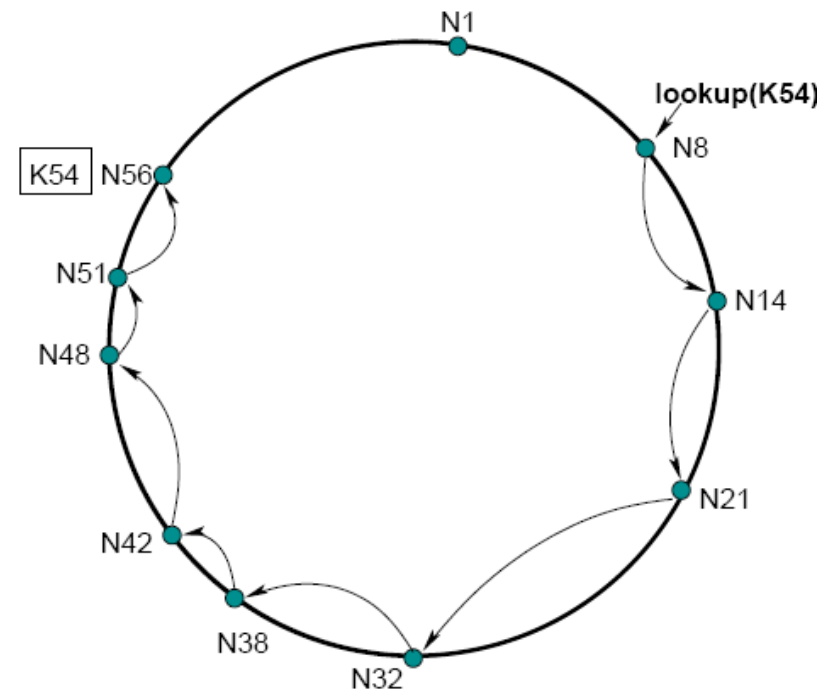
CHORD PRIMITIVE LOOKUP

Hash is **applied over a dynamic ring**

Lookup queries arrives from any node and **forwarded to the successor in one direction (one way)**

- Forward the query **around the circle**
- In the worst case, $O(N)$ forwarding is required
- By using both verses, can reach $O(N/2)$

In general $O(N)$



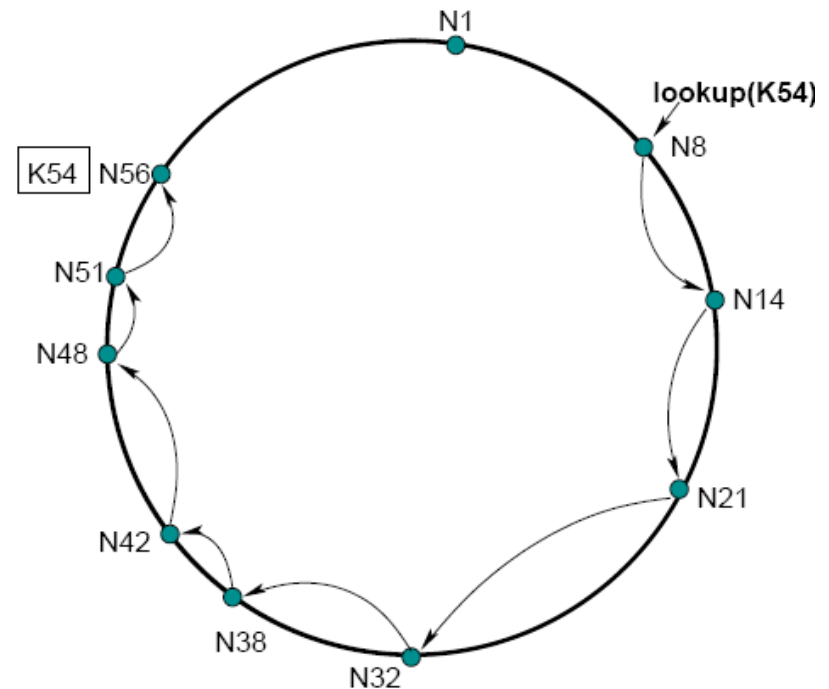
CHORD EFFICIENT LOOKUP

Lookup query can be fast forwarded in one direction skipping the vicinity

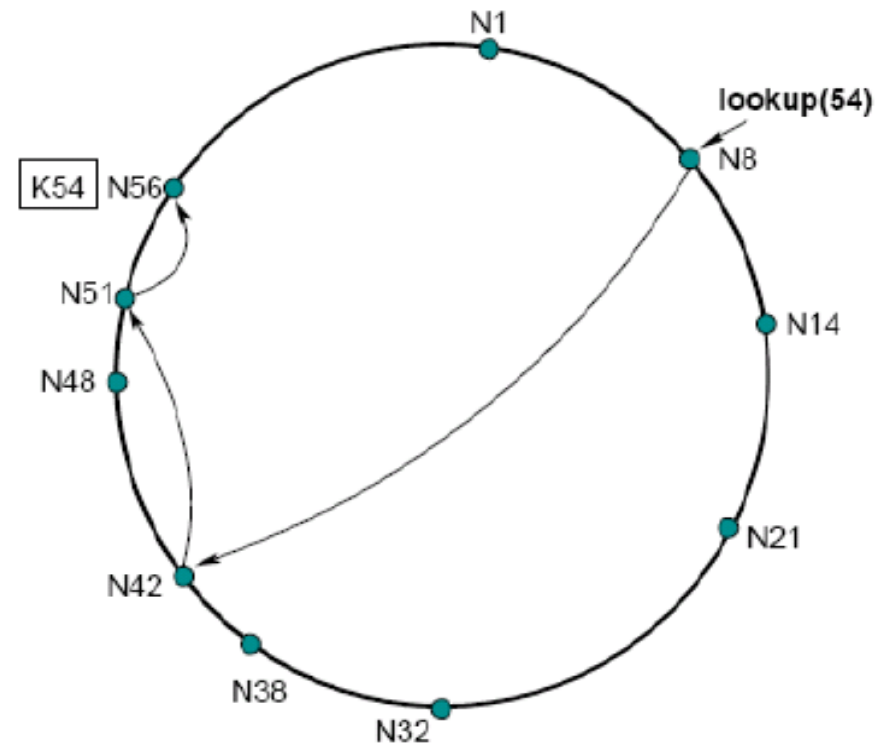
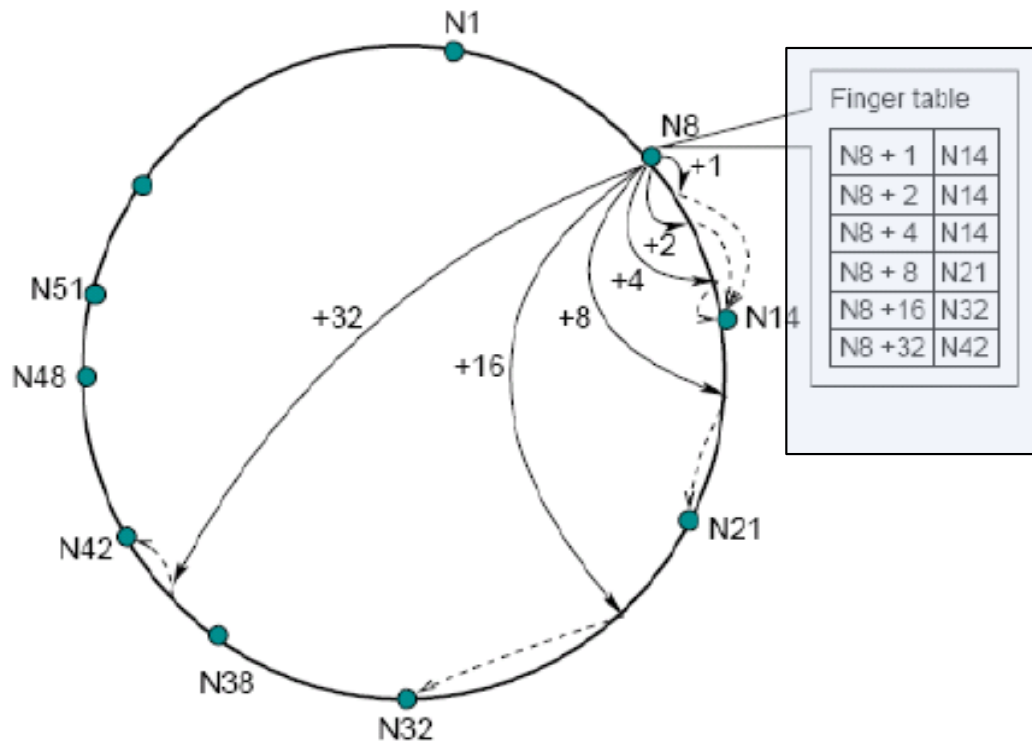
- For efficiency's sake by creating shortcuts in a data structure in any node

CHORD keeps **finger tables** to identify quickly far nodes (finger tables as caches for the successors at distance 2^i)

if a node join the ring, it have to create it's own finger table, but also every neighbour node have to update it's own finger table (same for predecessor and successor.



CHORD SCALABLE LOOKUP



The i_{th} -entry of a **finger table** points the **successor of the key** ($nodeID + 2^i$)

A finger table has $O(\log N)$ entries and the **lookup** is scalable and bounded to $O(\log N)$

CHORD CONSISTENT HASHING

CHORD works on the idea of making **operations easier**

Consistent hashing

Randomized

- All nodes receive roughly an equal share of load

Local

- Adding or removing a node involves an $O(1/N)$ fraction of the keys getting new locations

Cost of lookup via finger tables

- Chord needs to know only $O(\log N)$ nodes in addition to successor and predecessor to achieve $O(\log N)$ message complexity for lookup

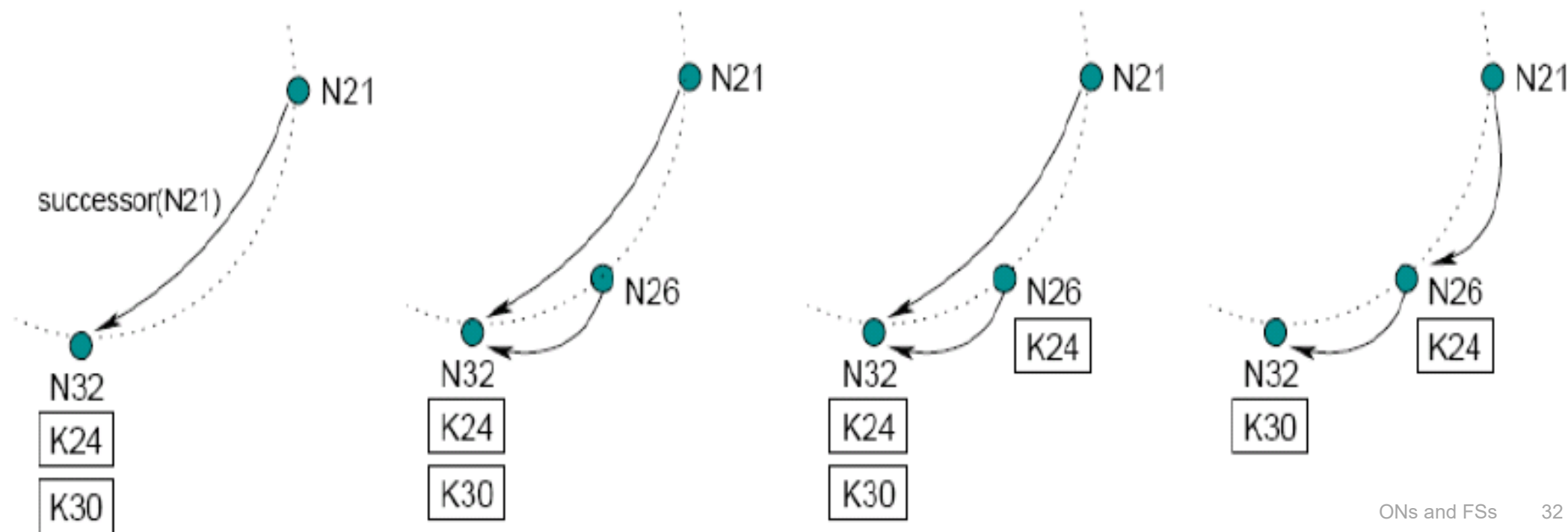
CHORD NODE JOIN

A new node has to

- Fill its own **successor**, **predecessor** and **fingers**
- Notify other nodes of which it can be a **successor**, **predecessor** and **fingers**

Simple way: find its successor, then stabilize

- Join immediately the ring (lookup works), then modify the structure organization – we will optimize **lazily** and **lately**



CHORD STABILIZATION

If the ring is correctly closed and set, then routing is correct, and **fingers are needed only for the sake of speed**

Stabilization

The support monitors the structure and organizes itself by controlling the ON freshness

- Each node periodically runs the stabilization routine
- Each node refreshes all fingers by periodically calling `find_successor(n+2i-1)` for a random i
- Periodic cost is $O(\log N)$ per node due to finger refresh

CHORD FAILURE HANDLING

The failure of nodes is handled by

Replication: instead of one successor, we keep a number of **R successors**

- **More robust to node failure** (one can find new successor if the old one failed)

In **robust DHTs**, keys replicate on the **R successor nodes of any node**

- The **stored data** become equally more robust

Alternate paths while routing

- If a finger does not respond, take the previous finger, or the replicas, if close enough

PASTRY

PASTRY is a DHT ON, like CHORD, differently organized for efficient access

Based on a **sorted ring in an ID space** (as in Chord)

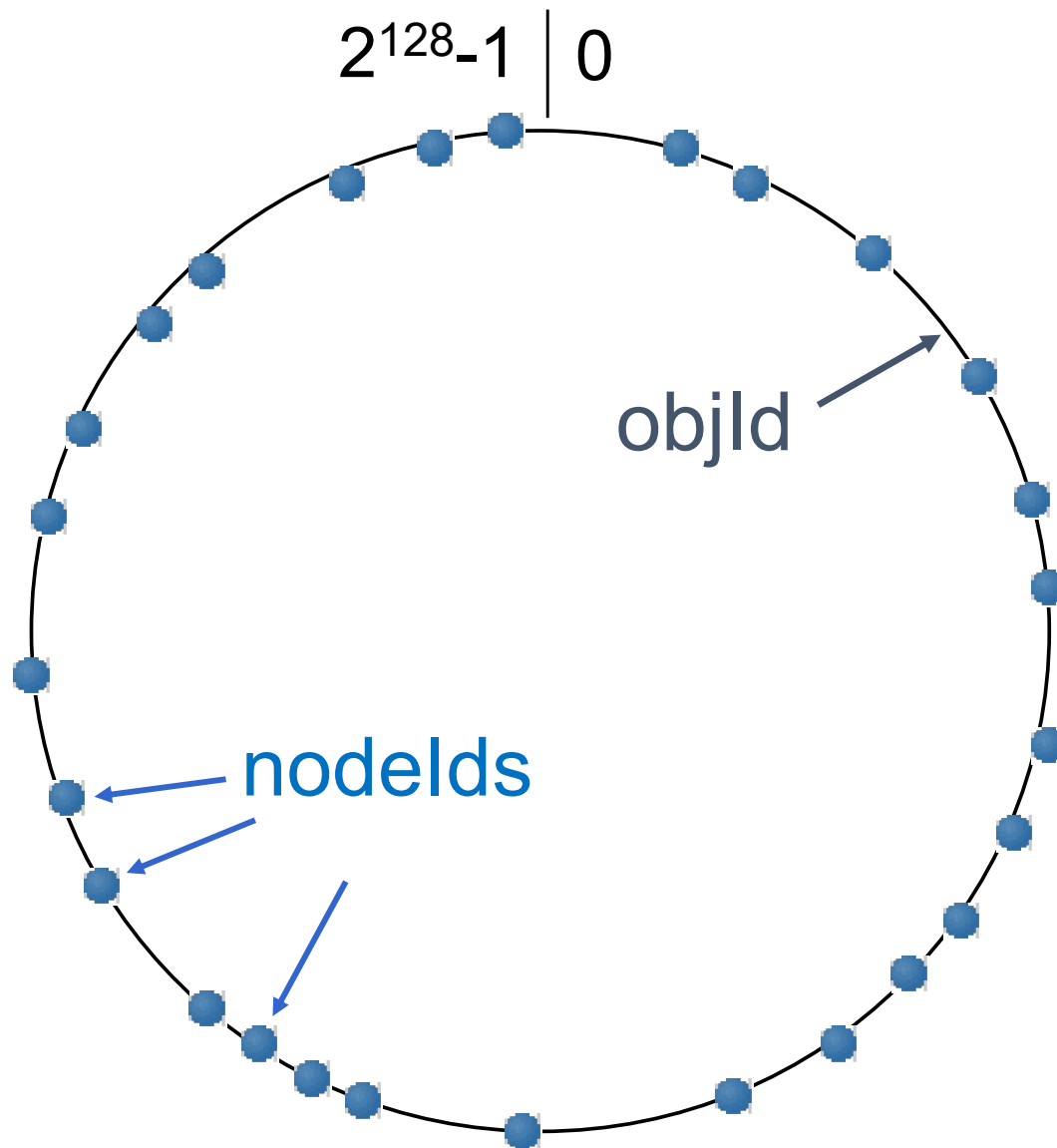
Nodes and objects are assigned a **128-bit identifier**

NodeID interpreted as a **sequence of digits in base 2^b (not bits)**

In practice, if the identifier **b=4** is used, names are **viewed as successions of digits in Hex (base 16)**

- **The node responsible for a key is the numerically closest** in hex starting from most significant **(not the successor)**
 - **Bidirectional sequencing by using numerical distance**
Routing tables **shortcuts** can speed up lookups
- **Nested groups the neighborhood for replication**

PASTRY: OBJECT DISTRIBUTION



**Consistent hashing of
nodes and objects ID**
128 bit circular id space

nodeids (uniform random)

objlds (uniform random)

**Invariant: nodes with
numerically closest
nodeid maintain objects**

PASTRY: OBJECT DISTRIBUTION

PASTRY keeps two tables

1. **Routing tables** (numeric neighborhood) to explore proximity and find numerically close neighbors
2. **Leaf sets** (vicinity) to maintain *IP* addresses of nodes with closest larger and smaller node IDs in the close neighborhood

Generic P2P location and a routing infrastructure

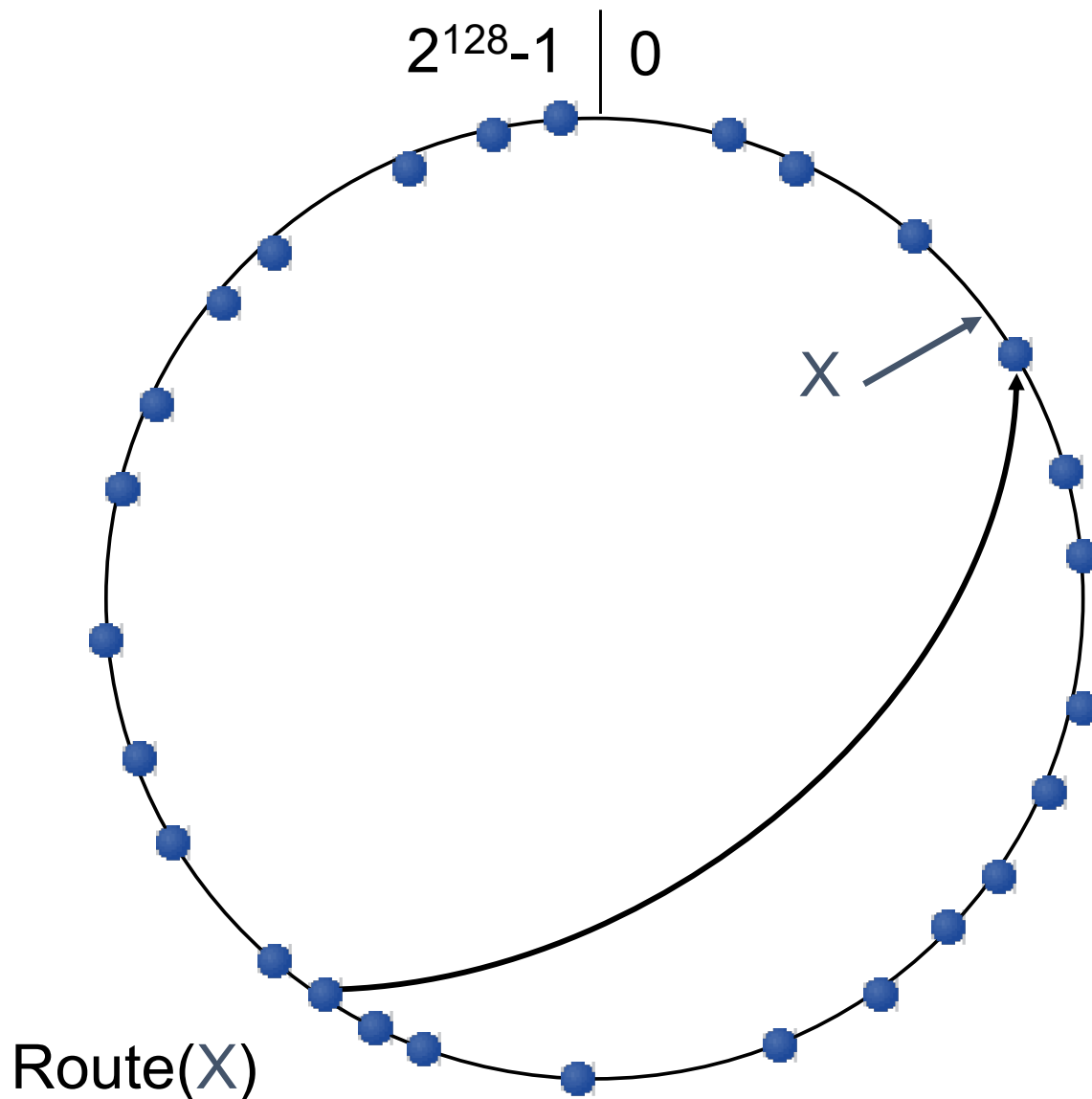
***Self-organizing* overlay network**

Lookup/insert object in $< \log_{16} N$ routing steps (expected)

$O(\log N)$ per-node state

Network proximity routing

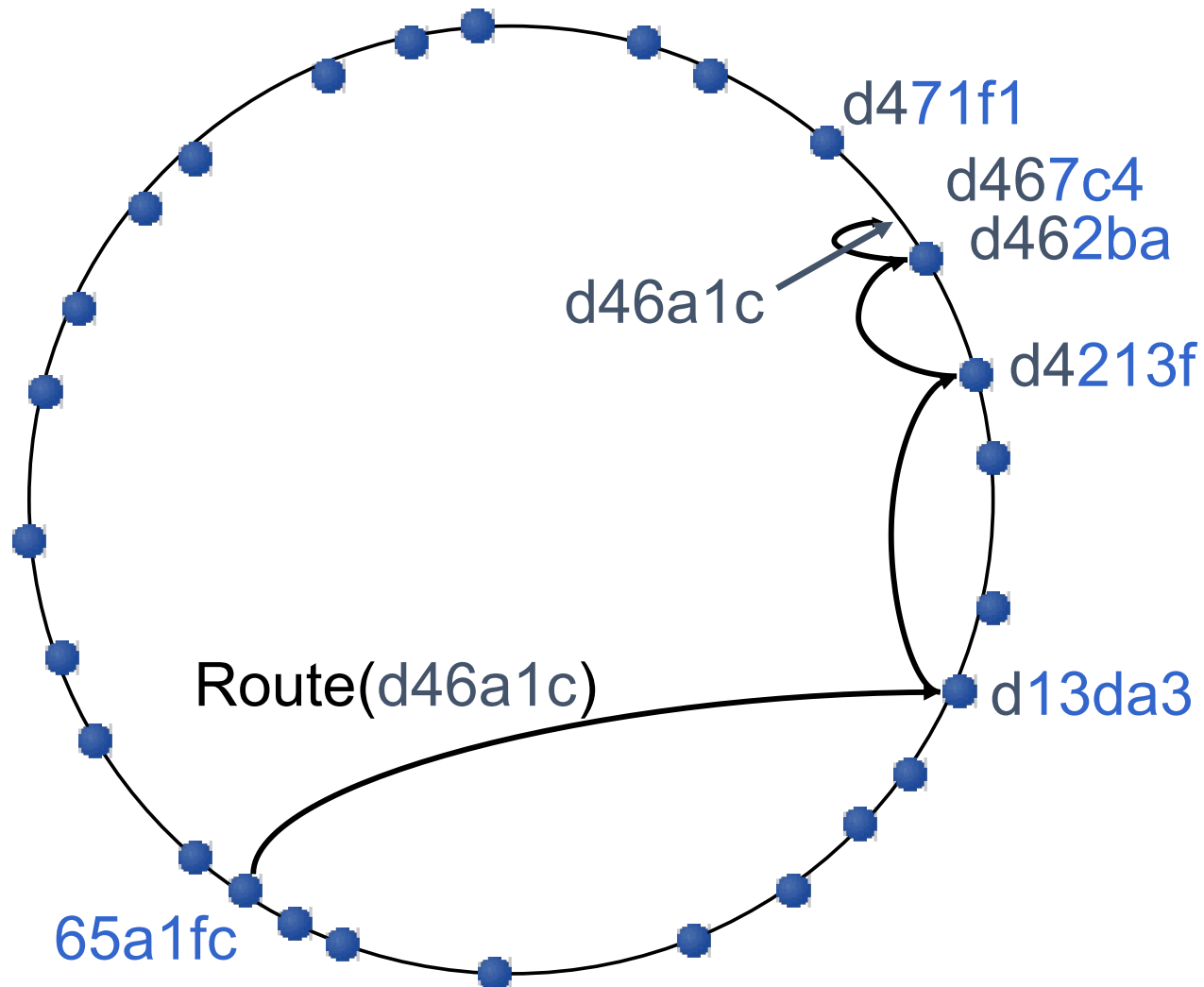
PASTRY INSERT / LOOKUP



A message with key X is routed to live nodes with nodeid closest to X

Problem: complete routing table not feasible

PASTRY ROUTING TABLES



Properties

$\log_{16} N$ steps
 $O(\log N)$ state

PASTRY ROUTING TABLES (# 65A1FC)

Row 0	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>		<i>7</i>	<i>8</i>	<i>9</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>		<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
Row 1	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>		<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>
	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>		<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>		<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
Row 2	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>		<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>
	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>		<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>
	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>		<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>		<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>
Row 3	<i>6</i>		<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>	<i>6</i>
	<i>5</i>		<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>	<i>5</i>
	<i>a</i>		<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>
	<i>0</i>		<i>2</i>	<i>3</i>	<i>4</i>	<i>5</i>	<i>6</i>	<i>7</i>	<i>8</i>	<i>9</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
	<i>x</i>		<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>	<i>x</i>

$\log_{16} N$
rows

PASTRY ROUTING TABLES

m=16

b=2

$b=2$, so node ID is base 4 (16 bits)

Contains the nodes that are *numerically* closest to local node
MUST BE UP TO DATE

ROUTING TABLES FOR QUERIES

Node ID 10233102

Leaf set

< SMALLER

LARGER >

10233033

10233021

10233120

10233122

10233001

10233000

10233230

10233232

Routing Table

02212102

1

22301203

31203203

0

11301233

12230203

13021022

10031203

10132102

2

10323302

10200230

10211302

10222302

3

10230322

10231000

10232121

3

10233001

1

10233232

0

10233120

2

Neighborhood set

2^{b-1} entries per row

13021022

10200230

11301233

31301233

02212102

22301203

31203203

33213321

Contains the specific nodes that are closest to the current node in the proximity metric for **REPLICATION**

Entries in the m^{th} column have m as next digit

n^{th} digit of current node

Entries in the n^{th} row share the first n digits with current node
[common-prefix **next-digit** rest]

Entries with no suitable node ID are left empty

PASTRY ROUTING TABLES AND LEAFSET

Leaf set

- **Set of nodes** that are **closer to the node**, the same as successors in Chord (vicinity for replication)
- L/2 smaller & L/2 higher
- **Replication boundary**
- Stop condition for lookup
- **Support reliability and consistency**

Routing table

- Provides **delegate nodes** in **nested groups** for numerical closeness
- **Self-delegates** for the **nested group** where the node belongs to
- $O(\log N)$ rows $\rightarrow O(\log N)$ lookup

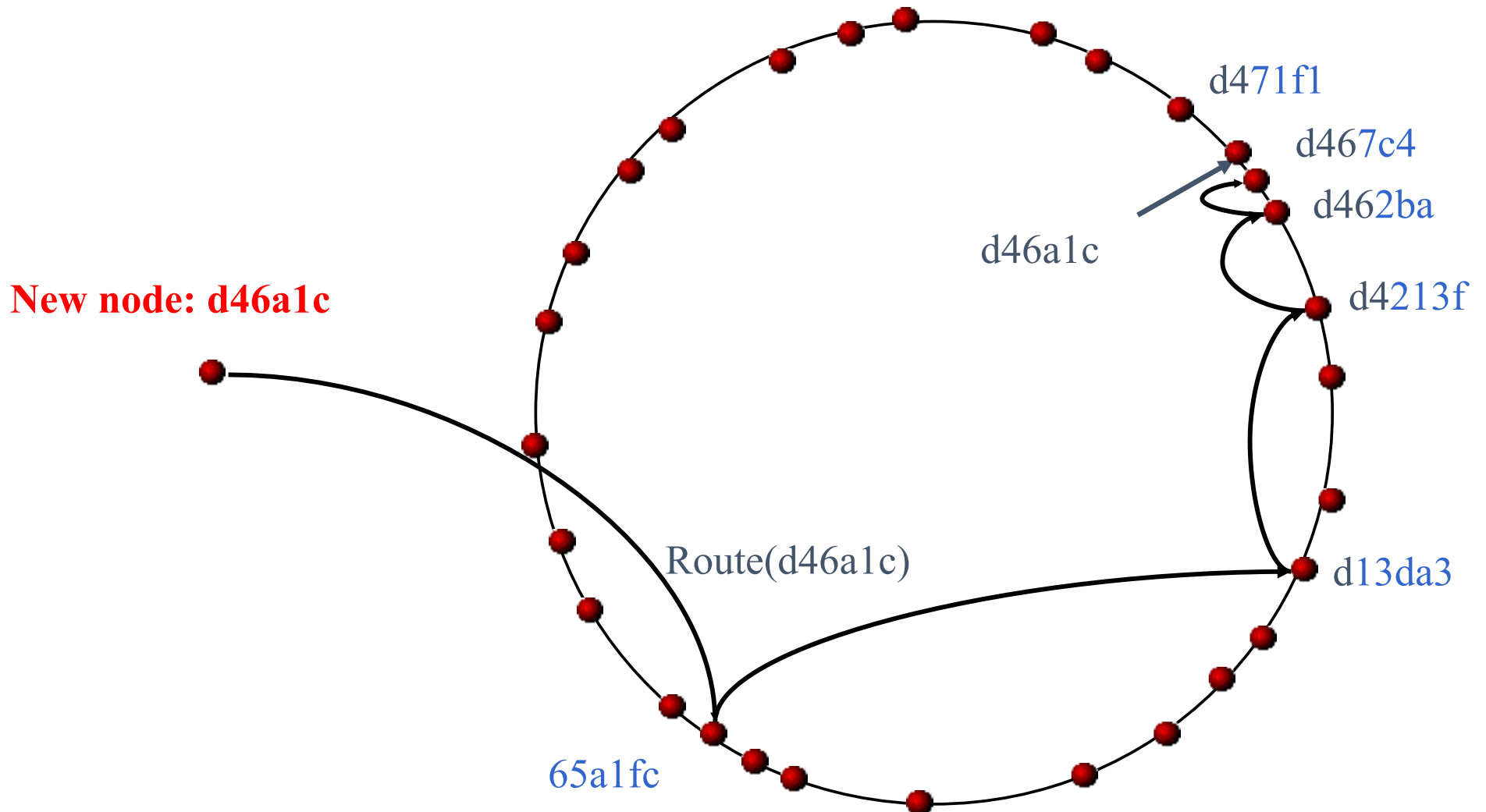
Base-4 routing table

NodeId 10233102

Leaf set	SMALLER	LARGER	
10233033	10233021	10233120	10233122
10233001	10233000	10233230	10233232

Routing table			
-0-2212102	1	-2-2301203	-3-1203203
0	1-1-301233	1-2-230203	1-3-021022
10-0-31203	10-1-32102	2	10-3-23302
102-0-0230	102-1-1302	102-2-2302	3
1023-0-322	1023-1-000	1023-2-121	3
10233-0-01	1	10233-2-32	
0		102331-2-0	
		2	

PASTRY NODE INSERTION



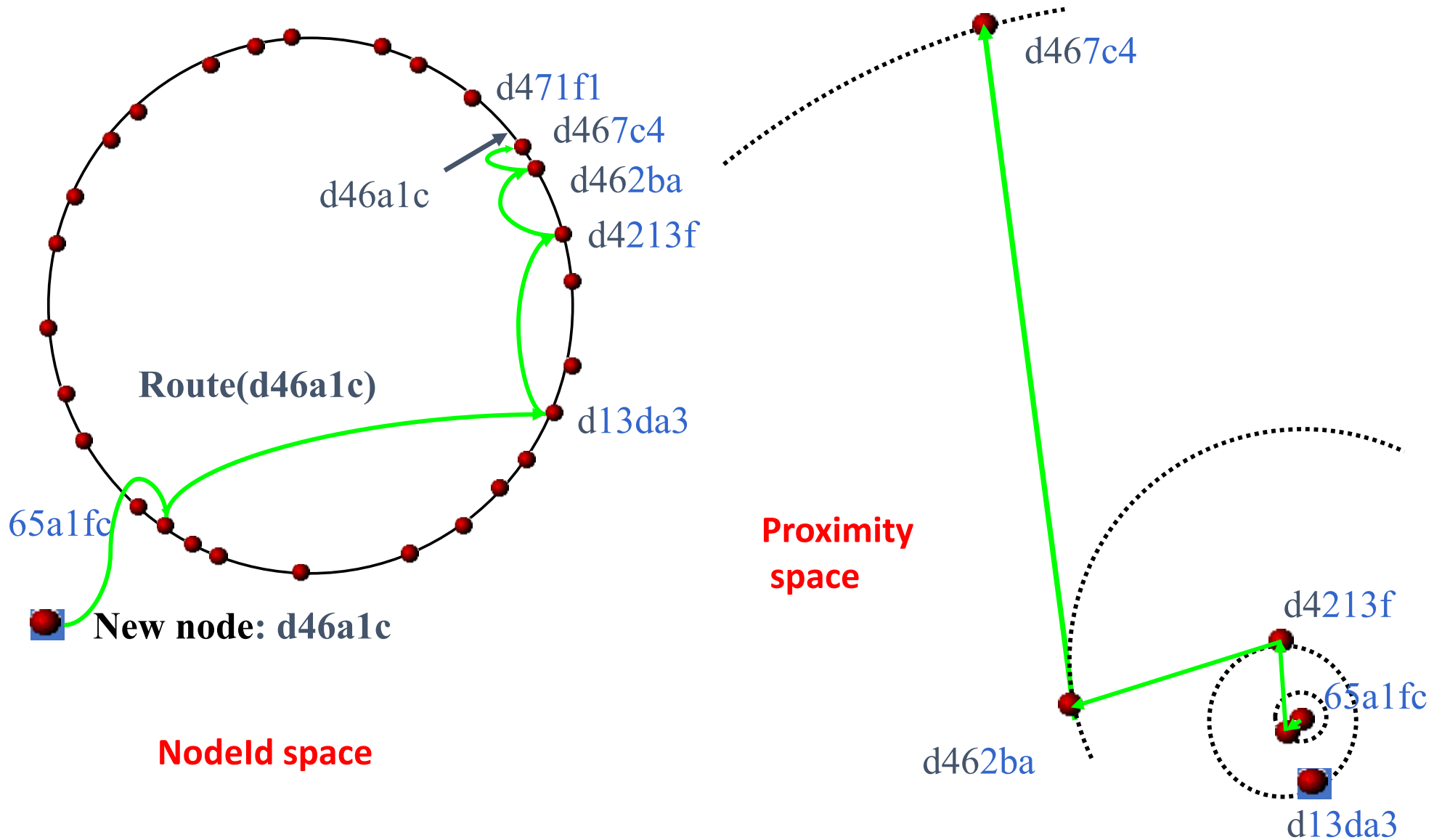
PASTRY JOIN & FAILURES

Join

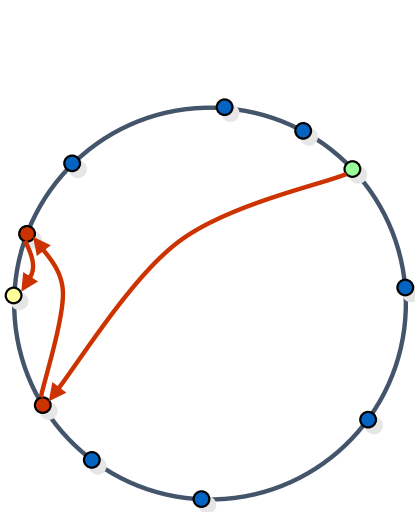
- Uses **routing** to find **numerically closest nodes** already in the network
- **Asks state from all nodes** on **the route** and initializes its own state

The operation is **efficient and smooth**

PASTRY NODE INSERTION

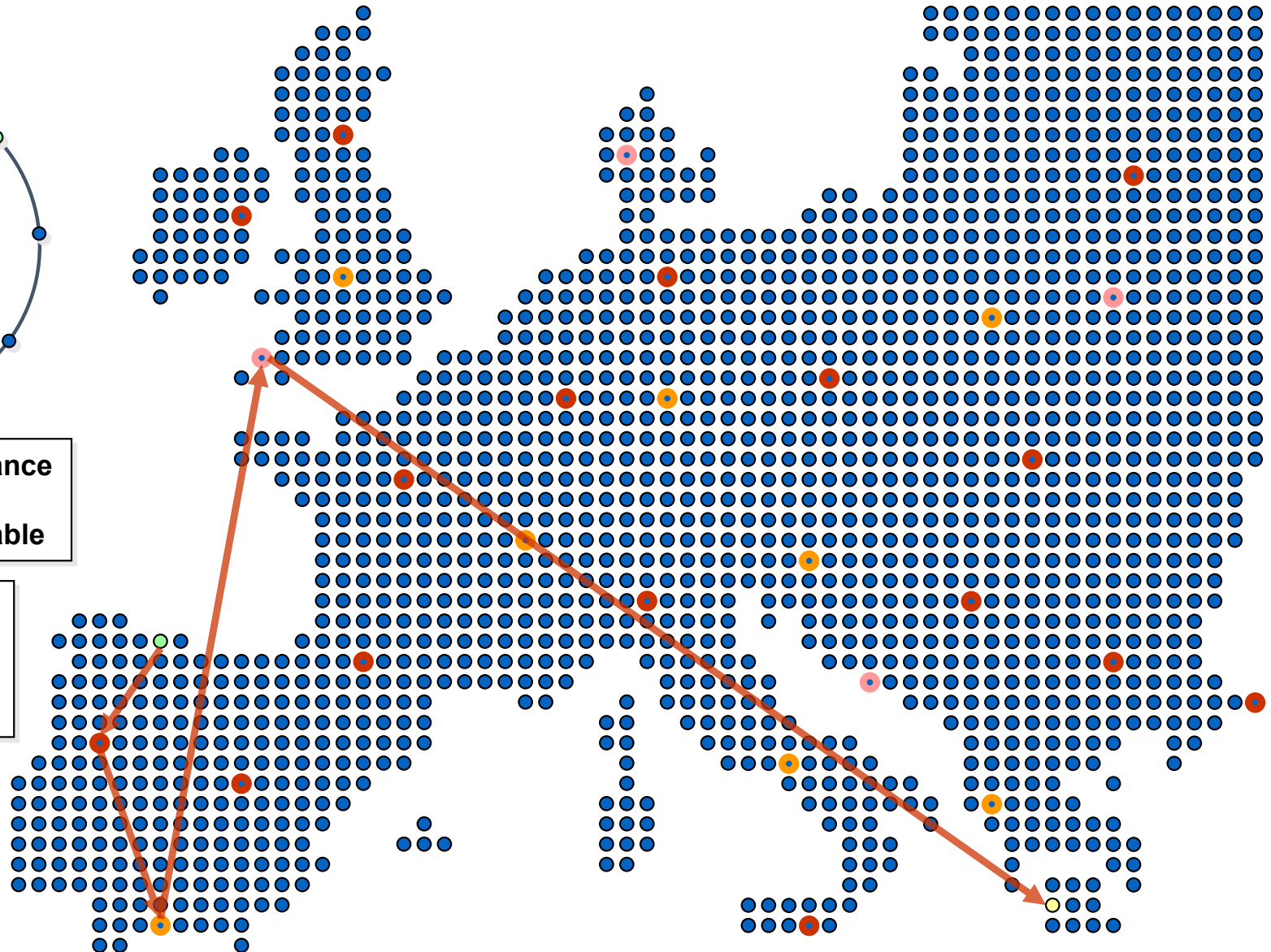


PASTRY ROUTING & TOPOLOGY



Expected node distance
increases with row
number in routing table

Smaller and smaller
numerical jumps
Bigger and bigger
topological jumps



PASTRY FAILURES

Leaf set members exchange keep-alive messages

Leaf set repair (eager):

Failed leaf node: contact a leaf node on the side of the failed node and add an appropriate new neighbor

Failed routing table entry: contact a live entry with same prefix of the failed entry until new live entry is found
if none found, keep trying with longer prefix table entries

Routing table repair (lazy):

get table from peers in the same row, then higher rows

OVERLAY NETWORK USAGE

ONs are very used inside P2P systems for file exchange

P2P (Napster, Gnutella, Kazaa, BitTorrent)

Social networks, for instance, need to connect fast different users:
Overlay Nets can help in preparing a support for those communications, ready to use and always available

So, inside the infrastructure you have those organizations dynamic and continually balanced

Social Nets (MSN, Skype, Social Networking Support)

Also in case of Cloud large infrastructure, to find parts of the support, when in need of finding new zones and copies

Cloud (for internal and federated discovery: **Cassandra**, **Zookeeper**,...)

 chord based

DISTRIBUTED FILE SYSTEMS

Network File System (or **NFS**) is the pioneer **C/S file system** and the most widespread network file system (even now)

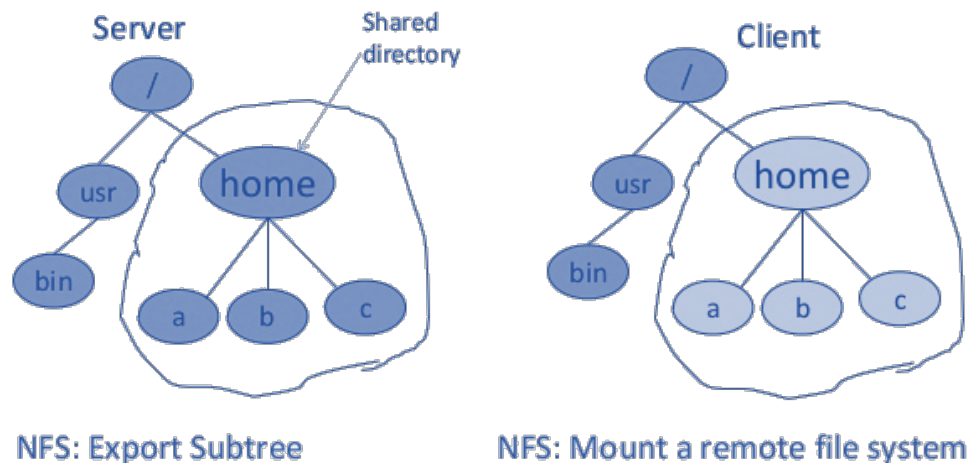
It is based on the idea of **client machines that interacts with server machines where files reside**

The implementation is **transparent** after mounting of file systems in any **client**. **NFS is stateless and efficient**: there is *no heavy weight on server machines*, while the load is on the client, and connections are **UDP**, etc.

There are many variations based on TCP connections, optimizations, etc.

NFS lack of any idea of Replication and QoS

NFS Overview



The clients 'mount' the shared directory, it becomes part of their own directory structure.

↳ but every client can mount the directories in different places.
It is not global

DISTRIBUTED FILE SYSTEMS

In **NFS** a **Client** can mount from several servers on its directories and deciding its visible configuration

So, the client file system can share different contents with several servers obtaining a peculiar global vision with different views

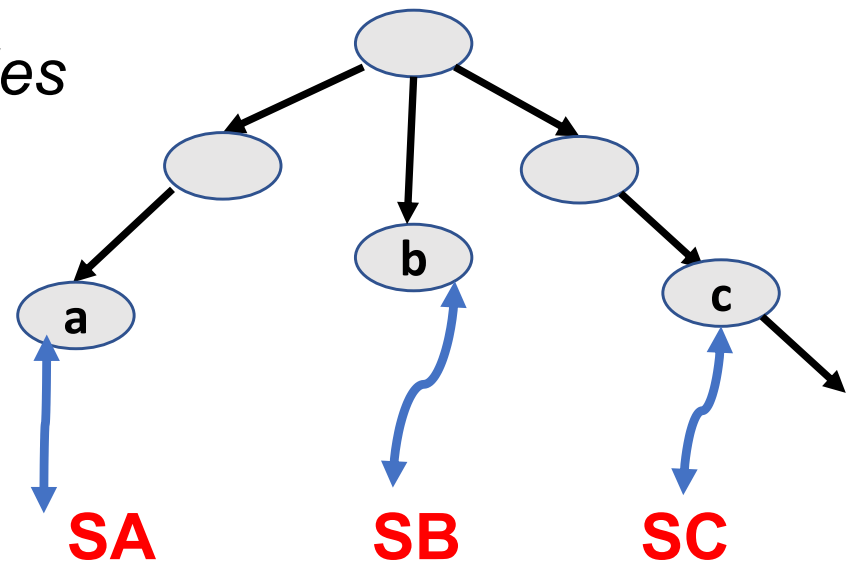
The **C** can mount its *a, b, c* directories from different servers

a from **SA**

b from **SB**

c from **SC**

from other file systems



NFS lack of any idea of sharing the same view of the same global file system among all nodes

NFS DISTRIBUTED FILE SYSTEM

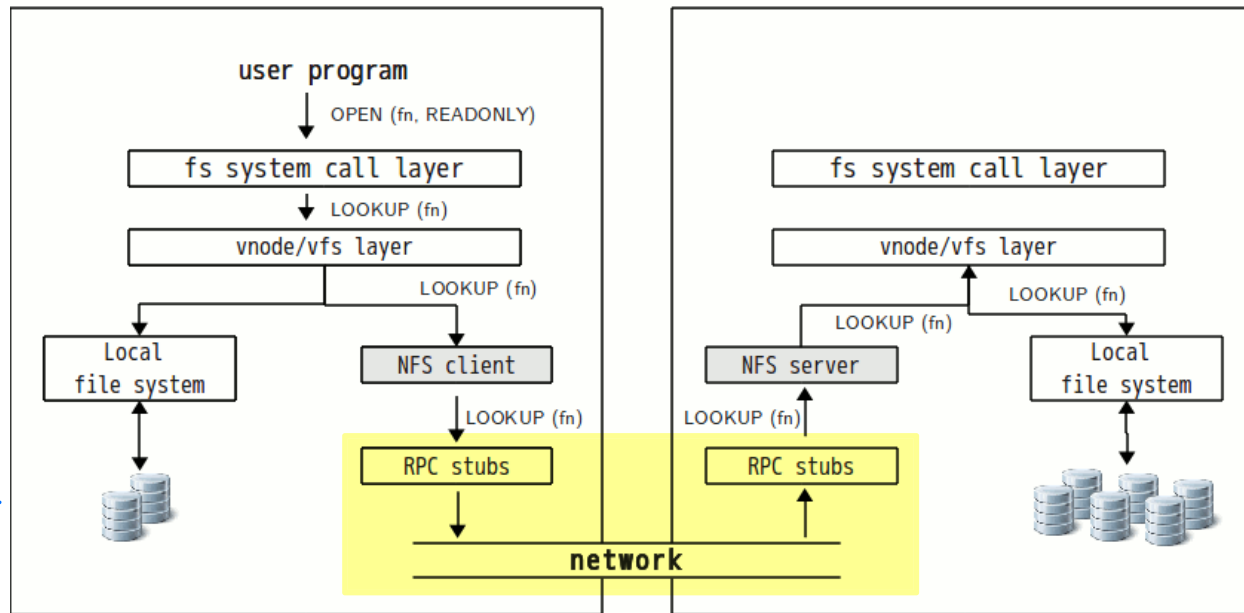
Network File System had the initial goal of using **RPC for the entire communication supports** so it strives for **efficiency and cost reduction**

The large diffusion is motivated by that choice

statefull ← client server → stateless

clients must keep track of operations on files

in this way if the server goes down, nothing is lost and it has not to manage every single client's state and operation.



The implementations are optimized and the overhead very low: the diffusion was incredibly large and still is

No replication nor QoS are granted **No global shared view**

MORE QUALITY FILE SYSTEMS

Other file systems were introducing **global quality**, in the sense of **replication**, and **uniformity** in view of the file

Andrew File System AFS

- A user can see the **same view** of the file systems, independently from where she/he is accessing
- Files are **replicated**, so that even if some nodes are not available the contents are always available
- Replication is **dynamically managed**, in general **the file is cached at the client site and more copies can be added** in case several clients are asking for the same contents (and deleted)
- The client uses a **call-back** to signal to the server any possible change actions *↳ generally one per writing*
- The clients can access from any possible OS support and several **additional services** are provided
- **Expected more reads than writes...**

GLOBAL FILE SYSTEMS

Modern **global systems** need **new tools** for **data storage with the necessary quality and also with global scalability**

File systems must use **replication** and other strategies toward **quality**

Starting with traditional **C/S FILE SYSTEMs** (similar to **NFS**) to typically, **dynamic management of data in all their parts to achieve better QoS**

Distributed file systems

- **Google File System** for Google data **GFS**
- **Hadoop file system** **HDFS**

→ global FS not NoSQL

Other modern solutions are based on different models and technologies ...

Later NoSQL

GOOGLE FILE SYSTEM (GFS)

GFS exploits Google hardware, data, and application properties to improve performance of **storage and search**

Large scale: thousands of machines with thousands of disks

Files are **huge** (normal files have **multi-GB** size)

- **Design decision:** difficult to manage billions of small files

File access model is **read/append (almost no write)**

- **Most reads are sequential**
- **Random writes practically non-existent**

New knowledge is appended to a file. Writings not on append only in case of wrong informations.

Component failures are '**normal**' events

- Hundreds of thousands of machines/disks
- **MTBF of 3 years/disk** → **100 disk failures/day**
- Additionally other failures: **network, memory, power failures**

MTBF – Mean Time Between Failure

DESIGN CRITERIA

Detect, tolerate, and recover from failures automatically

Deal with a “**limited**” number of **large files**

- Just a **few millions of large files**
- One file is **100MB – multi-GB**
- *Few small files*

Read-mostly workload

- Large **streaming reads** (multi-MB at a time)
- Large **sequential append operations**
 - Provide **atomic consistency to parallel writes** with low overhead
↳ every write writes on a block (completely)
⇒ one block & write request in parallel (in append)

Highly-sustained throughput much more important than **low latency**

DESIGN NOVEL STRATEGIES

Files stored as **chunks kept with their descriptions (metadata)** and **stored as local files** on Linux file system

Reliability through **replication** (at least **3+** replicas)

Single master coordinates access and keeps **metadata**

Organization and operations

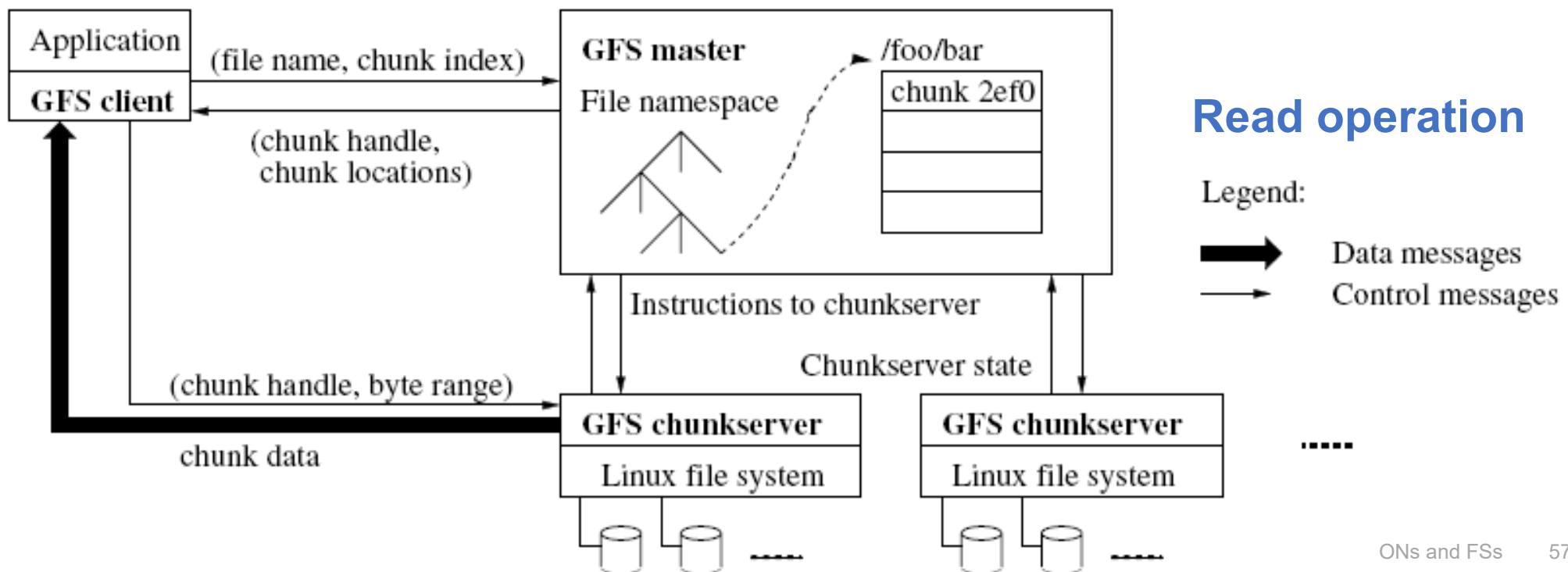
- **Simple centralized design** (one master per GFS cluster)
- Global knowledge to optimize **chunk placement and replication decisions** using **no caching**
- **Large data set/streaming reads** render **caching useless**
- **Clients cache meta-data** (e.g., chunk location)
- Linux buffer **cache** allows **keeping interesting data in memory for fast access**

DESIGN NOVEL STRATEGIES

One master server (*backups replicate its replicated state*) and **many chunk servers** (100s – 1000s) over Linux

- **Chunk: 64 MB** portion of file, identified by 64-bit, **globally unique IDs**
- Chunks are spread across racks for better throughput & fault tolerance
 → ~5 copies (max 10)

Many clients accessing files stored on the same cluster



MORE ON METADATA & CHUNKS

Metadata (the file description)

- 3 types: file/chunk namespaces, file-to-chunk mappings, location of replicas of any chunk
- All in memory (< 64 bytes per chunk) with GFS capacity limitation

Large chunks have many advantages

- Fewer client-master interactions and reduced size of metadata
- Enable persistent TCP connection between clients and chunk servers

OPERATIONS ON GFS

Normal operations are stream reads

They can go on in parallel and **can read any copy**

Rare operations are writes

They can be **stream append write**

(they can go on in parallel if they have a different space)

Even rarer operations are random overwriting writes

They can be done with some cautions

Control operations are metadata changes

Those are write operations, and must be **done sequentially**

Write operations are called **mutations**

RECORD STREAM APPEND SEMANTICS

The applications must deal with **record append semantics** for specific cases

Applications using ***record append*** should include **checksums** in writing records

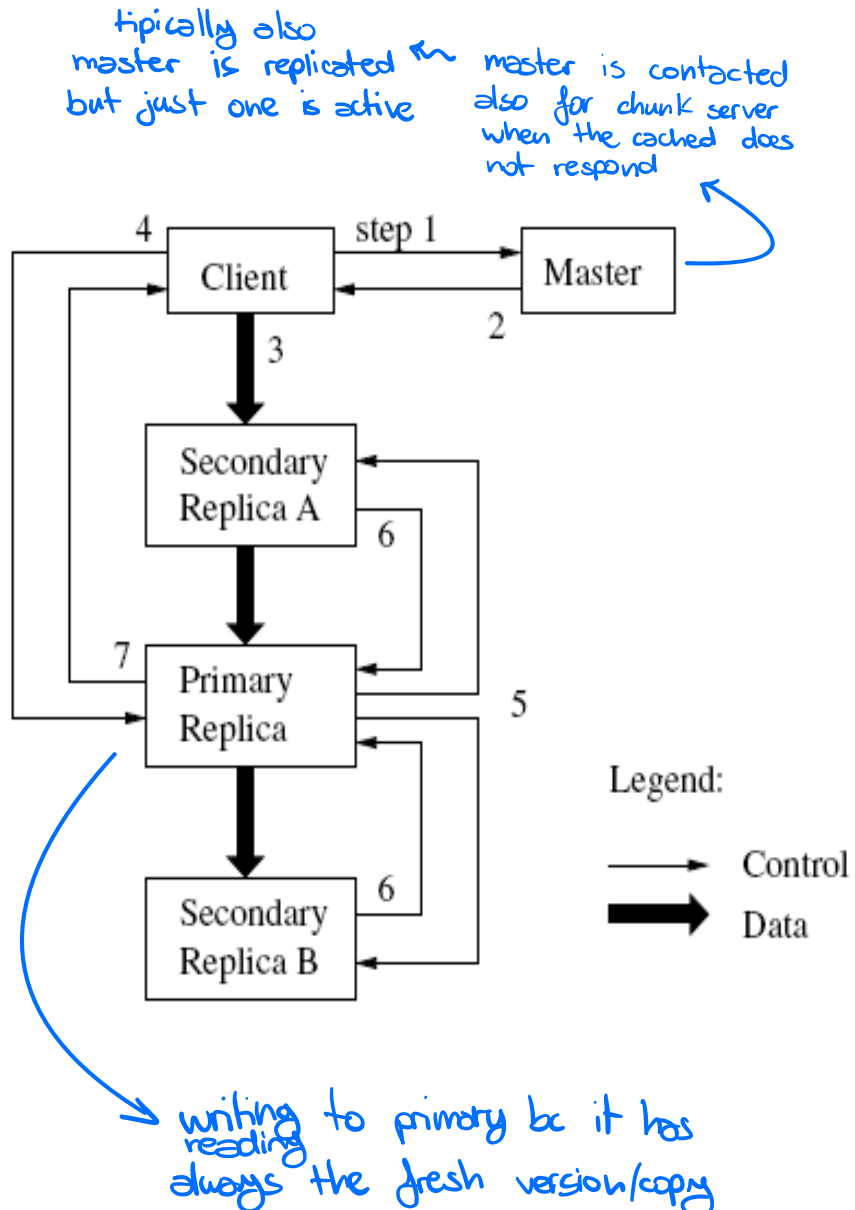
Reader can **identify padding/record fragments** using **checksums**

If application cannot tolerate duplicated records, should include **unique ID** in record

- Readers can use **unique IDs** to filter duplicates

in Unix primitives are atomic $\Rightarrow$ several writing on same portion overwrites each other $\Rightarrow$ not deterministic but no problem in concurrency
by semantic

STEP-BY-STEP WRITE



1. Identities of **primary chunk server** holding lease and the **secondaries** holding the other replicas
2. Reply
3. Push data to **all replicas** for **consistency** (see next slide for details)
4. Send mutation **request to primary**, which assigns it a serial number
5. Forward mutation request **to all secondaries**, which apply it according to its serial number
6. **Ack** completion
7. **Reply** (an error in any replica results in an error code & a client retry)

DATA FLOW FOR PIPE WRITE

Client can push **the data** to **any replica**

Data is pushed linearly along a carefully picked chain of **chunk servers**

- Each machine forwards data to “**closest**” machine in network topology that has not received it
- **Network topology** is simple enough that “distances” can be accurately estimated from IP addresses
- **Pipelining**: servers receive and send data at the same time

Method introduces **delay**, but offers good **bandwidth** utilization

MUTATIONS, LEASES, VERSION NUMBERS

Mutation: **write** operation that changes either the **contents** (write, append) or **metadata** (create, delete) of a chunk

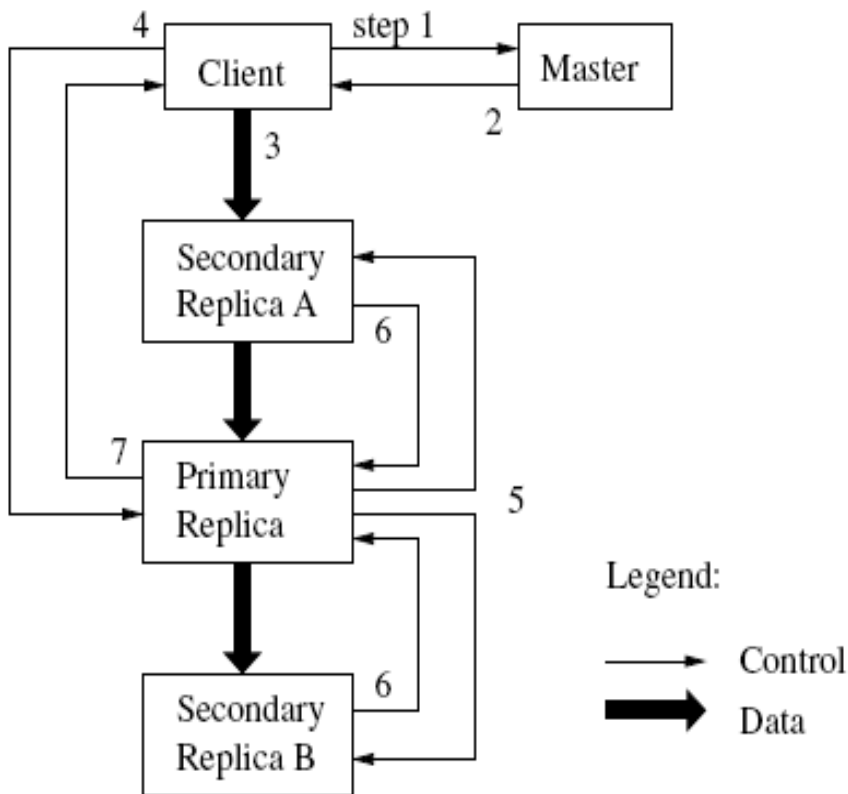
Lease: mechanism used to maintain **consistent mutation order across replicas**

- Master grants a **chunk lease** to **one replica (Primary chunk server)**
- Primary picks a **serial order to all mutations to the chunk** (many clients can access chunk concurrently)
- All replicas follow **this order when applying mutations**

Chunks have version numbers to distinguish between **up-to-date** and **stale replicas** and stored on disk at master and chunk servers

Each time master grants new lease, increments version & informs all replicas

STEP-BY-STEP MUTATIONS



1. Identities of **primary chunk server** holding lease and the **secondaries holding the other replicas**
2. Reply
3. Push data to **all replicas for consistency** (see next slide for details)
4. Send mutation **request to primary**, which assigns it a serial number
5. Forward mutation request **to all secondaries**, which apply it according to its serial number
6. **Ack** completion
7. **Reply** (an error in any replica results in an error code & a client retry)

CONSISTENCY MODEL

File **namespace mutations** (create/delete) are **atomic**

State of a **file region** depends on

- **Success/failure of mutations** (write/append)
- **Existence of concurrent mutations**

Consistency states of replicas and files:

- **Consistent**: all clients see same data regardless of replica
- **Defined**: consistent & client sees the mutation in its entirety
 - Example of **consistent but undefined**: initial record = AAAA
concurrent writes: _B_B and CC_C;
result = CBAC (none of the clients sees the expected result)
- **Inconsistent: due** to a failed mutation
 - Clients see different data function of replica

In case of **inconsistent data or undefined**, a data **reconciliation** must run

UNDEFINED STATE AVOIDANCE

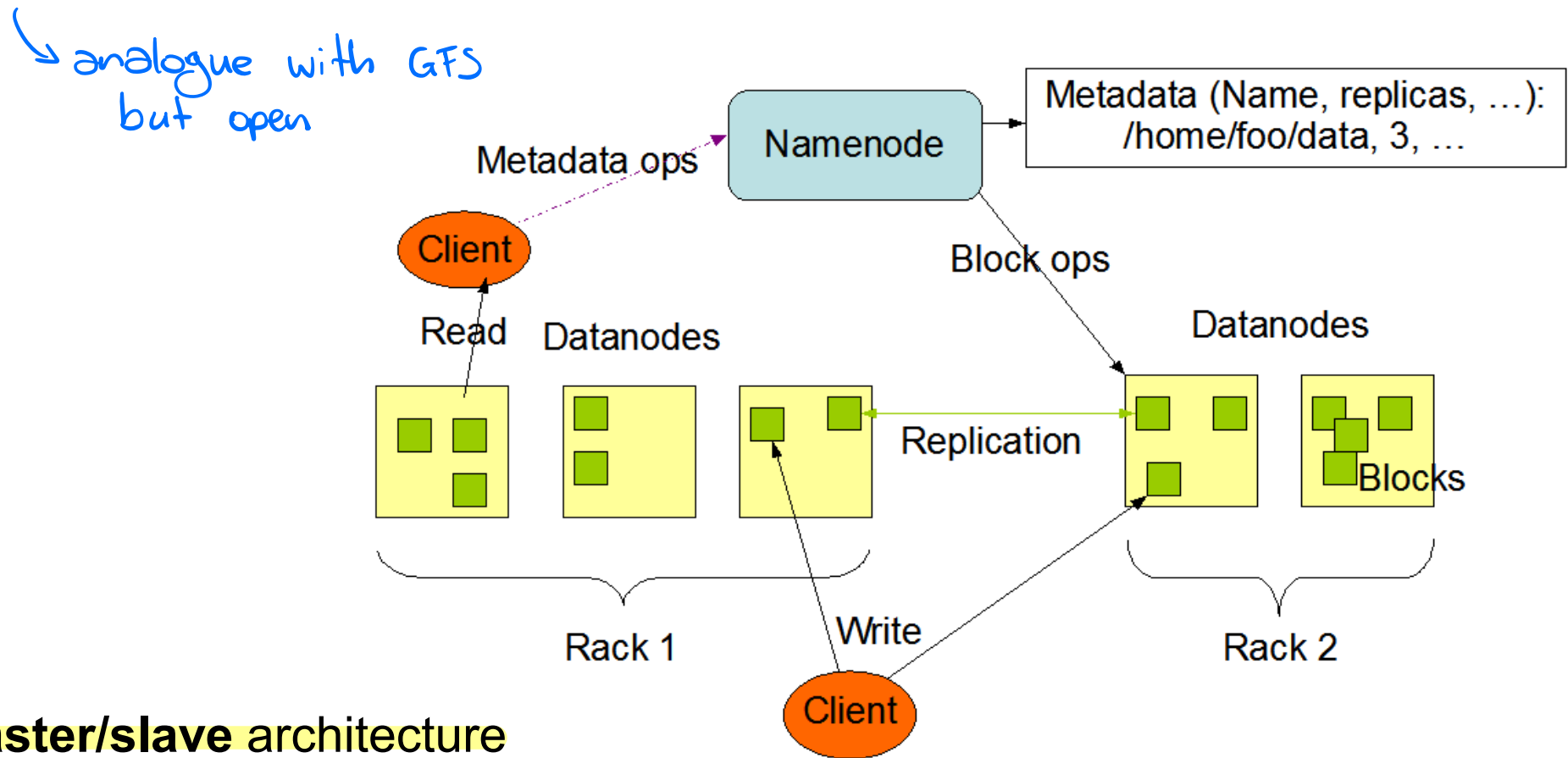
Traditional **random writes** would require expensive **synchronization** (e.g., lock manager)

- Serializing writes does not help (see previous slide)

Atomic record append: allows multiple clients to append data to the same file concurrently

- **Serializing append operations** at primary solves the problem
- The result of successful operations is well defined: **data is written at the same offset by all replica with an “at least once” semantics**
 - If one **operation fails at any replica**, the client retries:
as a result, replicas may contain duplicates or fragments
 - If not enough space in chunk, add padding and return error and Client retries

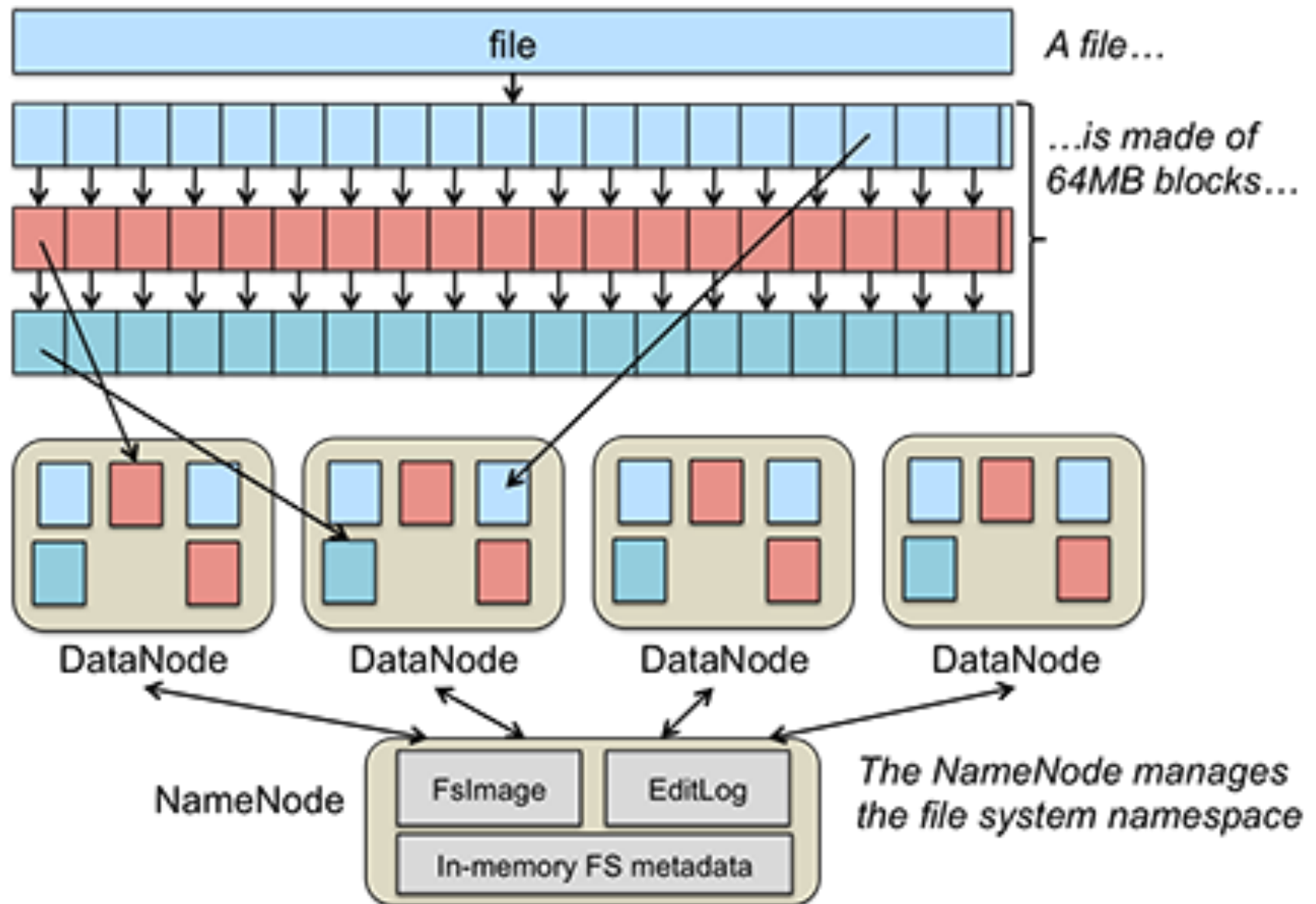
HDFS Architecture



- **NameNode** is the master (meta-data operations, access control)
- **DataNodes** are slaves: one copy per node in the cluster
- **Files** are stored in blocks in several DataNodes

APACHE HADOOP HDFS ACCESS

Again a replicated hidden support to maintain data and replicate them



ROLES AND PRINCIPLES OF HDFS

Hadoop Distributed File System is based on low cost hardware but with **high fault tolerance** and **high availability**

Applications access with **write-once-and-read-many** so the consistency model is similar to GFS and **computation is moved close to the related data** to operate upon

- **NameNodes** execute file system *NameSpace operations* like open, close, directories,...and decide on mapping
- **DataNodes** execute *read / write operations* requested from Clients and operates on block of data

HDFS is **written in Java** and must work on **normal hardware** to store **very large files** on different machines so to minimize the probability of faults by using replication

Any file decides its block size and its own replication degree

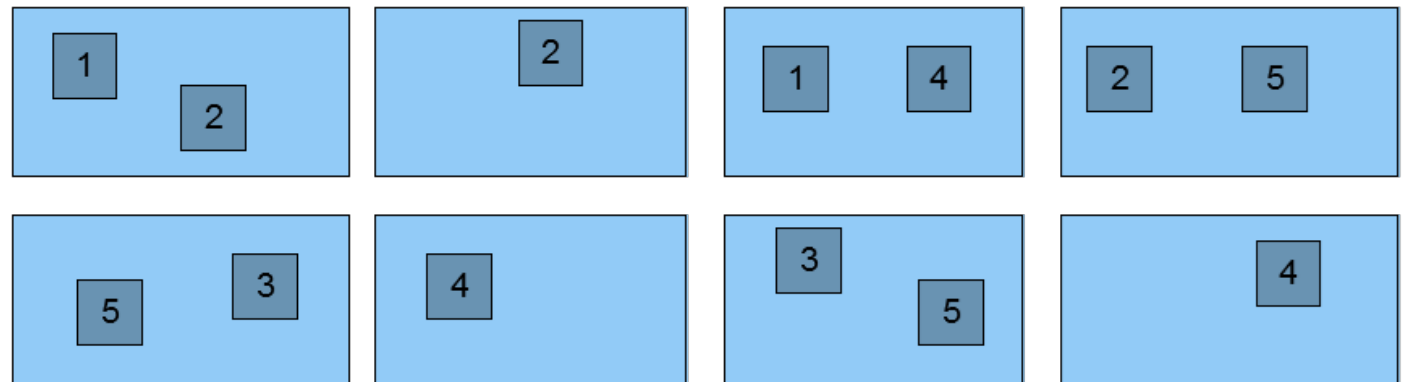
HDFS REPLICATION

Block Replication

Applications can
decide
dynamically
the **replication**
factor

Namenode (Filename, numReplicas, block-ids, ...)
/users/sameerp/data/part-0, r:2, {1,3}, ...
/users/sameerp/data/part-1, r:3, {2,4,5}, ...

Datanodes



Again master/slave architecture: **NameNode** receives **heartbeats** and **block reports** from **DataNodes**

- **Heartbeats** grant the **operation state** of DataNodes
- **Block reports** give the current block situations of DataNodes

PROPERTIES BEHIND THESE BRICKS

We have seen modern

Overlay Networks and Distributed File Systems

These are global tools to support new global systems

They are **black boxes** used by any global applications

They are **mechanisms** (bricks) to build other tools

They should favor some **system properties**:

- **Life cycle widening, scalability, dependability,**

Also try to favor strategic choices

- **Dynamicity** / staticity
- **Availability** / reliability
- **Eventual Consistency** / strict consistency of entities (ACID)
- **Fast answers** / safe delayed answers